The Open University of Israel
Department of Mathematics and Computer Science

Finding top-k flows in the data plane with and without an emphasis on the recent past

Project submitted as partial fulfillment of the requirements
towards an M.Sc. degree in Computer Science
The Open University of Israel
Department of Mathematics and Computer Science

By
Ozery Aviya

Prepared under the supervision of **Shir Landau Feibish**

October 2024

Abstract

Recent advances in programmable networks enable network operators to perform fine-grained analysis on *all* of the traffic as it traverses the network. One of the most common tasks, that has been widely studied, is to find the heavy hitter flows or the top-k flows. Several solutions have been presented that find such flows using a constant amount of memory and working within the confined resources and capabilities of the data plane. However, in order to pinpoint critical issues in the network, it is necessary to detect the flows that have been heavy in the *recent past*. Yet, existing approaches perform the task of heavy flow detection continuously, such that they can find the heavy flows over a boundless time period. In order to find heavy flows for shorter time intervals, sliding windows are often used. This requires multiple instances of the structure to clean out old data while still recording new data. Thus this requires a large amount of resources, and also requires predetermining the fixed length of the windows. We provide a formal definition of the *recent top-K* flows on a given stream, and present RecenTo a new deterministic counter-based algorithm for finding such flows in the data plane. The innovative technique used in RecenTo enables it to 'self-clean', thus making it easier for newer flows to enter the structure. RecenTo also makes use of a novel structure for maintaining the key and counter of the flows that provides support for the *mutual dependence* between them (i.e., sometimes the count is changed based on key and sometimes vice-versa). We evaluate RecenTo over continuous chunks of traffic and show that it consistently achieves a recall rate of ~ 0.9 for finding the top-k flows in each chunk.

Acknowledgements

First and foremost, I extend my deepest gratitude to my advisor, Dr. Shir Landau-Feibish, for her invaluable guidance, unwavering support, and the remarkable patience she has shown throughout my research journey. Her expertise and thoughtful mentorship have been instrumental in shaping this work, and her attention to detail has been a constant source of inspiration. Additionally, her assistance with the myriad of administrative tasks has been indispensable, and for that, I am truly thankful.

I am particularly grateful to Jonathan Diamant for his significant contribution to the conceptualization of this project. His clarity in defining the problem has been a cornerstone of my research, and his insights have greatly enriched the quality of my work.

Special thanks are due to Mary Hogan and Jonathan Diamant for their meticulous proofreading efforts and for offering their critical perspectives, which have greatly enhanced the final manuscript. Their willingness to share their expertise and provide constructive feedback has been of immense value.

To my husband, your unwavering belief in my abilities and your constant encouragement have been the bedrock of my perseverance. Your support has been a source of strength and motivation, making this achievement not just possible, but also enjoyable.

Lastly, but certainly not least, I owe a heartfelt thank you to my parents and my entire family. Your endless encouragement and the nurturing environment you have provided have been the foundation of my success.

This work is a testament to the collective effort and unwavering support of each individual mentioned, and it is with profound appreciation that I acknowledge your contributions. Thank you all.

Contents

1 Introduction **1**

2 Background **4**

2.1 Classic Solutions 4

2.1.1 Misra Gries 5

2.1.2 Space-Saving 5

2.2 Solutions In The Data Plane 6

2.2.1 Data Plane Limitations 7

2.2.2 HashPipe 8

2.2.3 PRECISION 9

3 RecenTo Design **10**

3.1 RecenTo High-Level Overview 10

3.2 RecenTo Counter Structure 11

3.3 RecenTo Register Updates 14

3.4 Handling Duplicate Keys 19

3.5 RecenTo improvements 21

3.6 The full algorithm 24

4 Implementation **26**

4.1 Data structures 26

4.2 P-registers update 29

4.3 Check for fully match case 30

4.4 F-register update 32
4.5 RecenTo logic 33
4.6 Recirculation decision 35
4.7 RecenTo resources 35

5 Emulation Implementation 37

6 Evaluation 39
6.1 RecenTo accuracy 40
6.1.1 Choosing The ***inc*** Parameter 40
6.1.2 Setting the Thresholds to Limit Recirculation 44
6.2 Finding the Recently Heavy Flows 45
6.2.1 Recall for different λ values. 45
6.2.2 RecenTo vs existing algorithms. 46

7 Conclusion 50

List of Figures

2.1 Programmable switch architecture [2] 6

3.1 Overview of packet processing in the pipeline 11
3.2 Handling partial *key* and *counter* 13
3.3 Key and counter updates. 17
3.4 Formation of duplicate entries. $inc = 2$ 20
3.5 Handling duplication 20
3.6 Insert case 22

4.1 P-register definition 26
4.2 P-registers and F-registers definition 27
4.3 Packet information 28
4.4 P register match action in P4 code 29
4.5 Set match case indication 30
4.6 Set replace case indication 31
4.7 Update F-register 32
4.8 Update F-register for second table 33
4.9 RecenTo logic 34
4.10 Resubmit decision based on the fix threshold 35
4.11 Resources summary as provided by P4i 36

6.1 Effect of table size on the accuracy 40
6.2 Effect of skew on the accuracy 41
6.3 Effect of place counter on the λ-recent-frequency 42

6.4 Effect of skew on the accuracy second bulk 43
6.5 λ-recent-frequency for various λ 46
6.6 RecenTo via PRECISION accuracy over time 47
6.7 RecenTo via PRECISION layout for last bulk 48
6.8 *ARE* for RecenTo-M vs. CMS and Dsketch. 49

List of Tables

6.1 *Insert threshold* for 128 counters per stage 44
6.2 *Fix threshold* for 1024 counters per stage 45

1 Introduction

In today's networks, many telemetry tasks require the detection of heavy flows (e.g. Heavy-Hitter detection or Top-K flows [10, 14, 16, 22, 23]). Heavy flow detection is used for a wide variety of tasks, such as traffic engineering [15], congestion detection [12], load balancing [20] or attack detection [21, 25]. Yet, in order to perform many of these tasks, it is necessary to find the flows that have been *recently* heavy. For example, bursty flows may cause congestion and packet loss. In order to identify these flows and restore network functionality, we need to identify the flows that have been heavy within the very recent past.

There are two prominent ways to track heavy hitters. The first is to use a sketch to keep a summary of the data [10, 14, 19], and the second is to maintain a fixed number of counters for the (hopefully) heavy flows [6, 18, 22, 23]. Most existing solutions for identifying heavy hitters usually run continuously and report flow counts accumulated over a long interval of time [7, 10, 14, 16–18, 22, 24], thus making them less suitable for detecting recently heavy flows. In sketch-based solutions, such as the Count-Min Sketch (CMS) [14], counters are repeatedly increased, so over time, if the structure is not cleared, the counters become very large. Thus the accuracy of the sketch will deteriorate and flow counts will become significantly overestimated (even for the small items). In counter-based techniques, such as PRECISION [7], over time counters of items maintained in the table will increase, keeping heavy flows in the table for a long time and making it harder and harder for new flows to enter the structure.

Recent advances in programmable networks enable adding new functionality in the data plane. However, while programmable networks make it *possible* to add new data plane functionality, programmable devices have extremely restrictive resources and processing capabilities and therefore adding this functionality is not easy. Nonetheless, different algorithms have been created that allow performing heavy hitter detection right inside the data plane [7, 17, 24]. Yet, techniques for finding *recently* heavy

flows in the data plane have been more limited.

In order to measure heavy hitters that appeared in the recent past, sliding windows are often used [5, 11, 12]. A fixed number of "windows", each with a separate instance of the structure, are maintained. Each "window" covers a fixed interval of time, so after an interval passes, the next window is used, in a round robin manner. One solution in which sliding windows were used is ConQuest [12]. There, sliding windows are maintained using a series of Count-Min Sketches [14], in order to find the heavy flows in a congested queue. While ConQuest provides a good flow count estimate for the time of a switch queue, it can not provide estimates for a longer period of time. Furthermore, only a handful of "windows" can be implemented within the programmable switch, thus limiting the granularity of the measurement interval. This example shows that sliding windows have some drawbacks: the first is the need to instantiate multiple instances of the data structure, which multiplies the required switch resources. The second, is that both the number of windows and the interval of time are fixed during compile time, making the solution rigid and less robust to changing network trends or needs.

In this work we present **RecenTo** (**Recen**t **To**p-k flows) a data plane algorithm for finding flows that have been heavy in the recent past. Instead of using predefined windows RecenTo implements an eviction scheme that evicts the heavy flows that were heavy a while ago but have been less frequent recently. Finding these flows is challenging, since the algorithm needs to not only identify that a flow is heavy but also *when* it was heavy, using a constant amount of memory. We make the following contributions:

- **Definition for the 'Recent Top-K' problem.** We provide a formal definition for the recent top-K flows in a given stream. The definition allows configuring the relevant part of the stream and also considers the timing of the appearances of the flows within the relevant part.
- **The RecenTo algorithm, a robust mechanism for identifying flows that were *recently* heavy in the data plane.** RecenTo is a deterministic, counter-based algorithm that works within the confined resources and limitations of the data plane. RecenTo maintains a table with a constant number of items (key and counters). In order to keep track of the recent heavy hitters RecenTo uses both *flow* counters that track the size of the flows maintained in the structure, and *place* counters that indicate when an item should be evicted from the structure. The *place* counters are incremented when a packet of the relevant flow arrives and decremented

otherwise. Thus, as the flow becomes less frequent, the *place* counter will gradually decrease. RecenTo uses this information to decide when a key can be evicted from the structure. In this way we can keep track of who is heavy at any given point, while still effectively measuring the flow's frequency.

- **Support for mutually dependent key and value.** The keys and counters in RecenTo are *mutually dependent*, meaning that in some cases the key is updated based on the value of the counters, and in other cases the counters are updated based on the value of the key. Due to the feed-forward processing of the programmable device, mutual dependence is not easily supported. RecenTo makes use of a novel structure which facilitates the mutual dependency with a very small number of packet recirculations. We believe this structure is also of independent interest.

- **Hardware implementation of RecenTo.** We have implemented RecenTo on the Tofino switch and show its limited resource usage. Additionally, we evaluate RecenTo and compare it to the state of the art algorithms. Evaluations show that RecenTo maintains high accuracy over time in detecting the recent heavy flows, outperforming existing solutions.

The remainder of this work is structured as follows: In chapter 2 we expand on some of the algorithms that exist today and that influenced the design of RecenTo, and also present limitations that apply to algorithms that are intended for the data plane. In chapter 3 we expand on the design of RecenTo, including corner cases. In Chapter 4 we describe the implementation of RecenTo for the Tofino programmable switch using P4. Finally, in chapter 6 we present the evaluation of RecenTo. There we analyze various parameters that affect the accuracy of our algorithm and also compare RecenTo to existing solutions.

This work has been accepted for publication in the PACMNET Journal and will be presented at the International Conference on emerging Networking EXperiments and Technologies (CoNEXT) 2024.

2 Background

Over the past few decades, big data processing has become a common task, requiring calculations to be performed on the information in an efficient manner. Consequently, many algorithms have been developed in order to provide a solution to various information processing problems.

Some of the most widely used and studied algorithms relate finding the most prominent items in a stream or the *heavy hitters* [6, 7, 13, 14, 22]. In this paper, we look at two variants of this problem: **Frequency Estimation**, i.e. for any given flow, estimate the size of the flow, and **Top-K**, i.e. identify the k largest elements within the stream.

In order to accurately answer the above problems, it would be necessary to save exact counters for each flow in the stream. However, given the long-tail distribution normally found in network traffic, the number of distinct flows can be very large, thus requiring huge amounts of information to be maintained, which is not practical. In order to reduce the amount of information maintained we must find more efficient ways to approximate the counters of the flows. Existing algorithms for flow counter approximation are mostly divided into two main categories: **counter-based algorithms**, which keep a counter per element, for a subset of elements from the data; **sketch-based algorithms**, in which a sketch is kept to approximate the counters for all of the elements, where one counter may be used by several elements. Counter-based algorithms have been both experimentally and analytically shown to provide more accurate counters for the worst-case [8].

2.1 Classic Solutions

We describe some of the most well-known solutions for heavy-hitter detection, which will establish the basis for the rest of our discussion.

2.1.1 Misra Gries

The Misra-Gries (MG) algorithm [23], is a counter-based algorithm, that provides a solution to both the frequency estimation and the top-k problems, using sub-linear space. The algorithm maintains a table T of w key-counter pairs, where it maintains a counter for a subset of the elements in the stream. In order to solve the top-k problem, MG will strive to keep only the heavy elements in the table with an appropriate counter for each. MG evicts the smaller elements from the table, in order to make space for the heavier ones. The algorithm works as follows: given a stream S of n items, for each item $x \in S$, if $x \in T$, MG increments its counter c_x by 1. Else, if there is an empty space in the table it will insert x into the table and set its counter to 1. Otherwise, the MG algorithm will decrease *all* of the the counters c_y of each $y \in T$. MG under-estimates the flow counts, and the error rate of MG is determined by the size of the table. For $w = 1/\epsilon$, the counter estimated by the algorithm will be at most ϵn below the real value[8].

Similarly to MS, a newer solution named Heavy-Keeper(HK)[18] also increases existing items, and otherwise decreases. Unlike MG, in HK d tables are maintained. Flow keys are hashed to determine the location of the flow in each table, and only the fingerprint of each flow key is maintained. Additionally, HK maintains a min-heap, where all the heavy items are kept with the full key and the counter. A flow is inserted into the min-heap only if its estimated counter is higher than the minimum counter in the heap.

2.1.2 Space-Saving

The Space-Saving algorithm (SpS) [22] is another counter-based algorithm for the top-k and frequency estimation problems. As in the MG algorithm, this algorithm also maintains a table T of w key-counter pairs, and works as follows: given a stream S of n items, for each item $x \in S$, if $x \in T$, SpS increments its counter c_x by 1. Else, if there is an empty space in the table it will insert x into the table and set its counter to 1. Otherwise, SpS finds the item y with the smallest counter in T and replaces it with x in addition to increasing its counter by 1.

Because SpS, unlike MG, does not reset the counters every time it inserts a new element, it will create an estimation that is an upper bound to the real count of the item (i.e. it will overestimate the count), as opposed to MG,

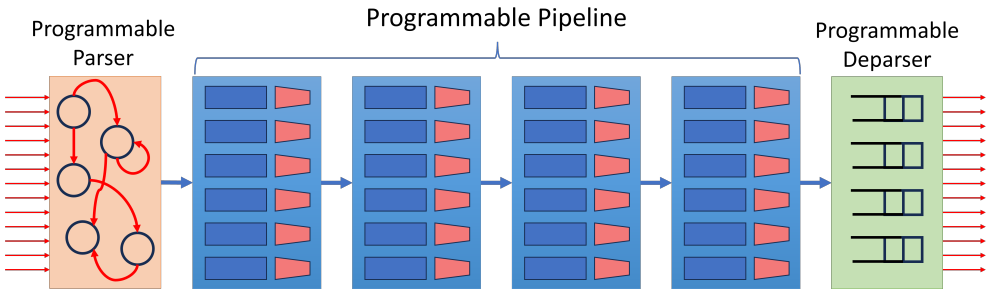


Figure 2.1: Programmable switch architecture [2]

which provides a lower bound for the actual count. Furthermore, studies show that the upper bound provided by SpS is usually closer to the real count[13].

A variant of SpS is RAP (Randomized Admission Policy) [6]. RAP is similar to SpS, yet in RAP a new item will not *always* be inserted into the table. Instead, the decision to insert the item will be made randomly, thus only some of the items will be inserted. The benefit of the random insertion is that it reduces the "noise" caused by the small items, that will only stay in the table for a short while. Additionally, it improves the counter accuracy of heavy items since they do not enter the table with a high initial count that may significantly overestimate the count.

2.2 Solutions In The Data Plane

Network programmability presents new opportunities for network analysis that can potentially process *all* of the traffic that traverses the programmable devices in the data plane. Specifically, we would like to perform algorithms such as the ones described above. The PISA architecture is presented in Fig.2.1 Each packet processed by the switch is first parsed, in order to extract the relevant header fields, after that the packet is processed by the programmable pipeline. The pipeline contains a fixed (normally $\sim$ 10-20) number of stages. Furthermore, in order to keep up with line-rate, there are extremely restrictive resources and processing capabilities in each stage. The packet traverses the pipeline in feed-forward manner, and once it exits the pipeline it is deparsed.

2.2.1 Data Plane Limitations

Programmable switches impose harsh constraints on both memory and packet processing, which pose significant challenges. Looking at PISA (Protocol Independent Switch Architecture) [9] these constraints include:

Limited processing. The programmable switch processing pipeline contains a number of stages and processes packets in a feed-forward manner, such that each packet is processed by each stage only once, with limited processing in each stage. the number of operations performed in each stage is limited, and even basic operations such as multiplication and division are not allowed.

Division of memory. Each stage is allocated specified memory (i.e., register array) that can only be accessed when a packet is processed in that stage. Due to the feed-forward manner in which packets are processed, each packet is processed by each stage only once, and therefore can only access memory allocated to that stage once. If the memory needs to be accessed again, the packet may be recirculated to be processed again by the pipeline. However, recirculation could reduce switch throughput and thus must be performed sparingly.

Limited concurrent memory access. There is a limited number of memory accesses that can be performed in each stage. Consequently, it is not possible to access *all* of the items in the array.

Dependency in memory accesses. While a small number of memory accesses may be performed in each stage, the order in which the memory is accessed is not determined by the programmer and may even be done in parallel. Thus dependent memory access is not supported in programmable devices, meaning that we can't modify one register based on the value of another register in the *same* stage.

Due to these limitations, algorithms such as MG and SpS cannot be performed as-is in programmable devices. For example, it is not possible to access the entire table. Thus it is not possible to decrement all of the counters in the table, as required by MG, or even to find the minimum counter as required by SpS and RAP. Furthermore, it is not possible to perform an update to an item in the table after the packet passes the pipeline. Therefore, it is not possible to update *all* of the items if the key wasn't found as required by MG, or to insert a new key, as required by SpS or RAP.

2.2.2 HashPipe

The HashPipe [24] algorithm was the first variant of SpS that was adapted to work within the limitations of programmable switches. As in SpS, HashPipe maintains a table T of w key-counter pairs. Recall that in SpS an item x may be found anywhere in the table. Yet, due to the limitations of programmable switches, the algorithm cannot access all w key-counter pairs when looking for an item x. Similarly the smallest counter may be found anywhere in the table. To overcome this limitation, HashPipe divides T into d hash-tables, which will each be stored independently in a different stage (one array in each stage, d stages in total). Furthermore, HashPipe uses d different hash functions $h_1(x)...h_d(x)$, one for each of the d stages, and every item x will be indexed to $h_i(x)$ in stage i. So in order to look for x, instead of w memory accesses, HashPipe will only require d accesses (one in each stage). Consequently, the algorithm only searches for the smallest counter among those d values accessed.

HashPipe performs as follows: for each packet, if the packet's key x is found in the relevant index in the first stage, increase the counter. Otherwise, insert x into the first stage with a counter value of 1 instead of the current key, and save the key that was removed and its counter in packet metadata (this is data that gets carried with the packet throughout the pipeline). HashPipe will carry this value and compare the counter being carried with the one found in each stage, so that it will maintain a rolling minimum. If the carried counter is larger than the one stored in the stage, the keys and counters will be swapped, and the key with the smaller counter will be carried to the next stage. Upon reaching the end of the pipeline, the stored value in metadata (i.e. the rolling minimum) is discarded. Additionally, if the carried key is found in one of the stages, or if the hashed index is empty, the counters will be aggregated and stored in the table, the key in metadata is cleared and the process will finish.

Notice that if HashPipe would traverse the entire table before deciding to insert the new item, it would need to recirculate the packet. Instead, HashPipe always inserts the key in the first stage. The problem with this process, is that if the value is also stored in a later stage, we get duplicates of the same key, since the value was already entered in the first stage, and is also stored in later stage.

2.2.3 PRECISION

PRECISION [7] solves this possible duplication using a random insertion process. As in HashPipe, PRECISION also uses a d stage hash-table and accordingly d different hash functions. But unlike HashPipe, PRECISION does not always insert an item in the first stage. Instead, as a packet is processed it both looks for the item across all d stages, and it also calculates the minimum counter seen. If the *key* is found, the related *counter* will be incremented. Otherwise, after checking in all d stages, PRECISION will *randomly* decide if the new key will be inserted, in which case it will replace the key with the minimum counter seen. Since this decision is made at the end of the pipeline, in order to insert the key the packet must be recirculated. Recirculations potentially reduce throughput so they can't be done for every packet. Instead, PRECISION will only recirculate when inserting the key, which is performed randomly based on the minimum value (i.e., $\frac{1}{min+1}$) found in the path of the packet.

3 RecenTo Design

In this chapter we will describe the RecenTo (**Recen**t **To**p-k flows) algorithm, a counter-based algorithm designed for the data plane. RecenTo makes use of a unique eviction mechanism which enables it to find both the recent top-k flows and the recently heavy flows in a stream. In this section we describe the RecenTo algorithm and data structure.

3.1 RecenTo High-Level Overview

The algorithm's main purpose is to maintain the *recently* heavy flows. In order to identify the relevant flows (i.e. heavy hitters) the RecenTo algorithm maintains a subset of the flows that pass through the switch. Similarly to the above algorithms, RecenTo maintains d hash-tables of (*key, counters*) tuples, and uses d independent hash functions. Each table contains x rows, such that the structure can store a total of $x * d = w$ keys. That is, RecenTo can only store a constant amount (w) of keys, so in order to find the *recently* heavy flows RecenTo needs to "clean" out the small or old flows, to make room for the new heavy hitters. In order to do this, RecenTo maintains two types of counters for each flow: the *place* counter, and the *flow* counter. Using these two counters, RecenTo will be able to both estimate the count of the flow and determine when the place should be freed to allow another key to be inserted.

Each packet that is processed traverses each of the d tables. In each table i, the packet is hashed to an index $h_i(key)$ (See Fig. 3.1). If the key is found in this index, the associated *place* counter is increased by a predefined value *inc*, and the *flow* counter is increased by 1. Otherwise (i.e., it is not stored in this index), if the location is occupied, only the *place* counter will be decreased by 1. If the location is free (i.e., the *place* counter equals zero), and the packet's id was not stored in a prior table, the packet's key will be stored in this location and the *flow* counter will be reset and then

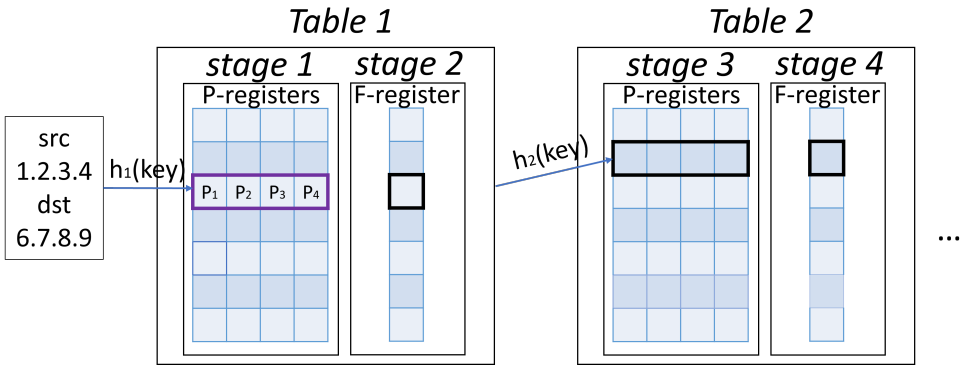


Figure 3.1: Overview of packet processing in the pipeline

incremented to 1. Using this process, RecenTo replaces the flows that are no longer relevant, on-the-fly, as it is processing the packets, so it does not require an additional cleaning process.

Notice that keys may be duplicated in several tables. This may occur when the key already exists in a certain table, but a free space was found in a prior table and therefore the key was inserted there as well. As we will describe in detail in Section 3.4, the second instance will be deleted. However, if the *flow* counter in the second instance is above a predefined threshold, the packet will be recirculated, so that this flow count may be added to the *flow* counter in the new location of the key.

Intuitively, RecenTo looks for flows that were recently heavy. Such flows will usually be characterized by having both a high *place* counter and a high *flow* counter. As time passes, if the flow was not heavy in the recent past, its *place* counter will gradually decrease until eventually the counter reaches zero. Once a *place* counter has reached a value of zero, it indicates that the place is empty, and another flow can take this empty place. In this way RecenTo clears the way for new flows to be counted by the structure. Note that the flow frequency estimation remains intact as long as the key was not replaced.

3.2 RecenTo Counter Structure

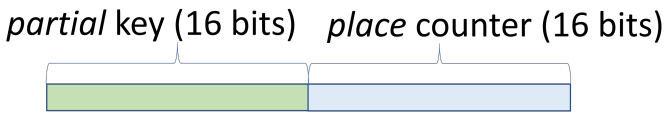
The RecenTo structure should support the two operations we described earlier. On the one hand, it needs to allow checking the *key* value first, so it can decide whether to increase or decrease the *place counter* (and increase the *flow* counter if there is a match). On the other hand, if it needs to

find an empty space, so that it may insert a new key, it needs to check the *place counter* first to see if it is equal to 0. As we mentioned in Section 2.2, dependent memory accesses cannot be performed in the same stage, and need to be placed in separate stages. Yet RecenTo sometimes needs to first look at the key and other times it needs to first look at the *place* counter. We call this a *mutual dependency* between the *key* and the *counter*.

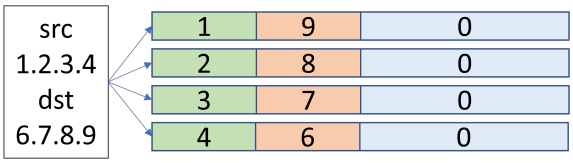
Splitting the key to deal with mutual dependency. In order to deal with this *mutual dependency*, instead of accessing the key and the *place* counter separately, RecenTo will maintain parts of the key and its *place* counter *jointly*, in the same register, so they may be accessed simultaneously. While dependency between registers is not supported, a register may be modified based on its own prior value (i.e., test-and-set), so a write to a register may be dependent on reading the *same* register. In this way, the *place* counter can be updated according to the key, and the key can be replaced when the *place* counter is reset, so both dependencies may be supported simultaneously.

In order to jointly maintain both key and *place* counter, we conceptually divide a single register into two parts: a *place counter and a* key (or parts of it). In our discussion we assume that two bytes are used for the *place counter*, and the rest of the bytes will be used for saving *part* of the *key*. So, for 32–bit registers, the *key* will also be allocated two bytes. The layout is shown in Fig. 3.2a. If the key is composed of $2c$ bytes, c registers are needed to store all of the information. Each *place* counter refers *only* to the *partial key* stored in the same register (we call this a P-register i.e., Partial key and its place counter register), and is not aware of the values stored in the other registers. Furthermore, since each P-register is only associated to *part* of the key, it will be increased by *any* flow matching this part of the key. This creates a much larger number of flow collisions (compared to when the *complete* key is matched). To limit the affect of collisions, the key should be split so that it represents as much information as possible from the full key.

As an example for this split, we examine a key made up of *(src IP, dst IP)* (in IPv4 protocol). Since 4 bytes are required for each IP address, a total of 8 bytes is needed to store the key. In enterprise networks, for example, it makes sense that several requests will be received from different sources to the same destination, and vice versa. Additionally, the address range of the first entry of any IP may also be narrower (for example in the CAIDA traces we used, many of the flows are mapped to the same flow space). We therefore took the following actions to reduce the amount of collisions:



(a) P-Register layout, 16 bits for *place counter* and others for *partial key*



(b) Storing 8 bytes of key in 4 P-registers with 4 associated *place counter*s

Figure 3.2: Handling partial *key* and *counter*

First, each register contains both a part of the *src IP* and a part of the *dst IP*. Such that each register is assigned one byte of the *src IP*, one byte of the *dst IP*, and also holds two bytes of the *place counter*. Additionally, as shown in Fig. 3.2b, the first entry of the *src IP* is maintained in the same register as the last entry of the *dst IP*, to reduce access collisions due to limited flow-space.

While P-registers allow dealing with mutual dependency, they cannot give reliable information about the real counter of the flow for several reasons: First, due to the flow collisions described above even flows with *partially* similar keys may increase the related *place counter*. Second, the *place* counters are affected by the reductions performed by the algorithm which may reduce the counter accuracy of active flows. Third, since *place* counters are limited to two bytes, they may only go as high as 0xffff. Note that in order to prevent overflow, once reaching this value, the counter will no longer be increased and will remain at the maximal value, until it is decreased by other items, allowing it to be increased again if needed.

The *flow* counter. In order to provide a more accurate estimation, RecenTo maintains the *flow* counter. This counter will be kept in a full register in a different stage (we call this a F-register i.e., Flow counter register), so it's not limited to 0xffff. Furthermore, it will be placed after the P-registers containing the split key, and thus it will only be increased if *all* the parts of the key were matched. Note that the *flow* counter will not be decreased in case of a mismatch, so that it will provide a better approximation of the true count of the flow. The only case in which the flow counter will be reset, is when the key (some or all parts of the *key*) changes. Notice that since this counter is in a consequent stage, it can't be used by the

algorithm to decide if the location is free or not, since that decision needs to be made earlier. However, the information maintained by the P-registers *is* sufficient for showing that the place counter is reset, and this is the critical information needed before updating the *key*.

That is, the place counters can give some indication of how heavy the flow is at a given time. If both the place and the flow counters are high, the flow is *recently* heavy. If only the flow counter is high, this is a flow that was heavy in the past and may be evicted from the table soon. If the *place* counter in any P-register is zero the *partial* key is replaced and the flow counter is reset. If both the *place* and the *flow* counters are small, this is probably a newly inserted flow, and it is not yet known if this will be a heavy flow or not. Finally, if only the place counters are high (one or more) this is usually caused by partial key collisions or duplicates (we will expand more on how this is handled later in this section).

So far we have described the data in one row of the RecenTo table, that required $c + 1$ registers, c P-registers to store the *key* parts, with the related *place counter* (e.g. for the *key* source-destination pair in IPv4, $c = 4$), and 1 F-register to store the *flow counter*. Note that the P-registers can be stored in one or more stages (as there is no dependency between them), but the F-register must be stored in a later stage (as it is incremented or reset based on all the other P-registers).

We note that the solution shown assumes a switch architecture with a register size of 32-bits (as it is more inhibiting). Our method can easily be adapted to 64-bit registers as well. In addition we look at a flow key comprised of the source-destination IP pair. In IPv4 the key contains a total of 8 bytes, but the solution can easily represent keys with less or more information (e.g. 12 bytes key for IPv6 protocol) and will require a corresponding number of register accesses and stages.

3.3 RecenTo Register Updates

The RecenTo data structure is made up of d tables maintaining registers as described earlier. For each packet that arrives, the flow key is hashed via d uniform hash functions $h_1, ..., h_d$, which map the packet to a single index in each of the d tables.

Adapting the counter decrement model to constraints of the data plane. Recall that MG also performs an eviction scheme. For each item, if it was in the table, its counter was incremented. If it was not, and there was an item

with a count of zero, the new item would be inserted. Otherwise, all of the counters in the table would be decremented. Due to the constraints imposed by the programmable hardware, and specifically, the limited number of concurrent memory accesses allowed, RecenTo cannot access *all* of the items in the table, and thus cannot handle all of the counters. Instead, RecenTo will only refer to the counters of the indexes that it accesses. In addition due to the feed-forward processing in the programmable switch, in order to decrement the counters *only* when the current flow is not in the table (i.e., *after* searching through the entire table), the packet would need to be recirculated, causing a very high recirculation rate. Therefore the counters will be decremented in *any* case of a key mismatch, although it may be found later in the table. Note that if the key was already found in a prior table, the counter will also be decreased in order to get uniformity between the decrement process in all stages in the case of a mismatch. Counters will also be decrement if the key matches again, in order to avoid duplicates.

Varying counter increments. Since RecenTo decrements the *place* counters in *any* case of a mismatch, many packets might inadvertently decrease the *place* counters, causing them to quickly reset. In order to compensate for this, when there is a match, RecenTo will increase the *place* counters by more than 1. Specifically, RecenTo defines the *inc* parameter to indicate the value by which *place* counters are incremented when there is a match. The right value of *inc* can be informed from variants such as the distribution of the traffic, and the number of items in the RecenTo data structure. We expand further on how the value of *inc* should be chosen in Section 6.1.

Updating the P-registers (key and *place* counter). In Algorithm 1 we present the details of the process described above as it is performed in a single table i, for a single P-register j. Please note that bytes [0,1] in register j contain the relevant bytes of the *partial key*, and bytes [2,3] contain the relevant bytes of the *place counter*. We also point out that in lines 2 and 3 before RecenTo increases the *place counter*, a check is made that no overflow occurs, otherwise the maximum will be set. To summarize, the algorithm will do the following: if the *partial* key matches, the *place* counter will be increased and *partial key matched* will be reported (lines 2-7). If there is no match *and* the *place* counter equals zero the partial key will be inserted, and *replaced* status will be reported (lines 8-10) (Note that the *place* counter is not changed, meaning that small flows with only 1 packet will not be counted. There is an option to increment the *place* counter from the first packet of a flow, but this is not needed, and in fact may introduce noise.). In any other case that the *partial* key does not match, the *place* counter will

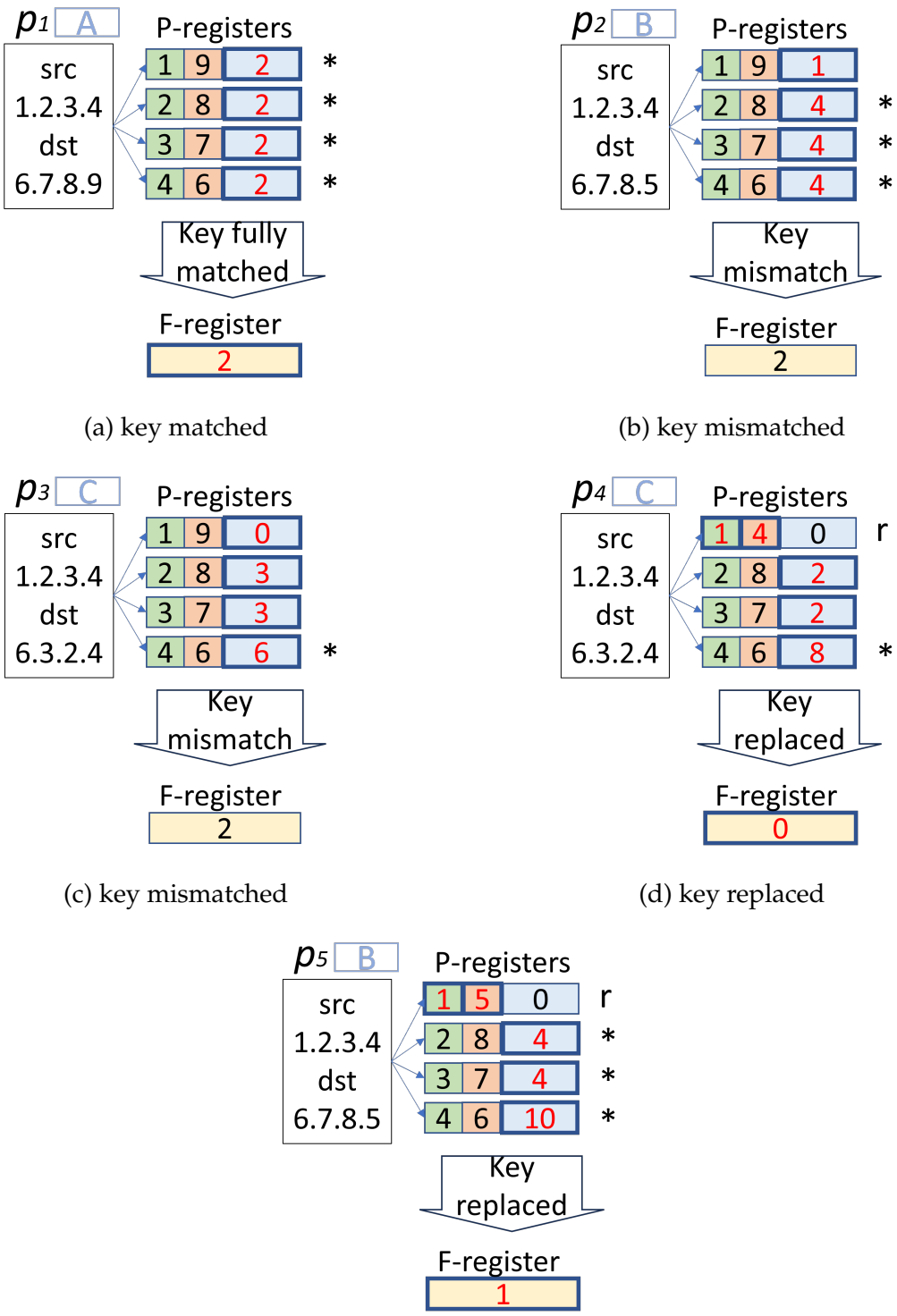
Algorithm 1 update_p_register(h_i, j, $partial_key$)

```
if reg_{i,j}[h_i][0,1] = partial_key then        #key in register matched new key
  if reg_{i,j}[h_i][2,3] + inc > 0xffff then      #overflow - set maximum
    reg_{i,j}[h_i][2,3] ←0xffff
  else                                           #Increase place counter by inc
    reg_{i,j}[h_i][2,3] ← reg_{i,j}[l_i][2,3] + inc
  end if
  Mark key[j] as updated                         #report partial key matched
else if reg_{i,j}[h_i][2,3] = 0 then             #place counter empty
  reg_{i,j}[h_i][0,1] ← partial_key              #set the partial key
  Mark key[j] as replaced                        #report partial key replaced
else                                             #decrease place counter
  reg_{i,j}[h_i][2,3] ← reg_{j,i}[h_i][2,3] − 1
  Mark key[j] as mismatched                      #report partial key mismatched
end if
```

be decreased, and *key mismatch* status will be reported (lines 11-13).

Updating the F-register. After all of the P-registers have been processed, all parts of the key have been checked, and we can go on to update the F-register. Notice, that when the key was compared against the keys in the P-registers, there could have been three cases:

- **Key fully matched.** If *all* parts of the *key* matched, this packet's flow is definitely stored in this index. Thus the associated F-register holds the flow's counter and the counter is increased.
- **Key replaced.** If at least one of the parts of the key had been replaced, then the flow stored by the P-registers has been changed and thus the flow counter should be reset to 0, as the old information is not relevant to the *newly* stored key. In the case that at least one part of the key (i.e., at least one P-register) was replaced and all others already match, the key is fully matched and we can start to count this new stored flow, and the counter in the F-register will be set to 1.
- **Key mismatched.** If at least one of the parts of the *key* was not matched, then the packet's flow is definitely not stored in this index. Note that parts of the key can match due to a collision with other flows, nonetheless this is not a full match and thus the flow counter should not be updated.



(a) key matched

(b) key mismatched

(c) key mismatched

(d) key replaced

(e) key replaced and matched

Figure 3.3: Key and counter updates. $inc = 2$; (*) represents a partial key match; (r) indicates key has been replaced.

In-depth example. Fig. 3.3 shows an example of how the above process is performed for a sequence of packets, showing how P-registers and the F-register are handled by RecenTo for a single index of one table. Note that in this example we assume that $inc = 2$. Suppose that the index started in an initial state as shown in Fig. 3.2b.

In Fig. 3.3a, packet p_1 of flow A is received. The relevant part of A's flow key is compared to each of the associated P-registers accordingly. As can be seen, all of the key parts match, and therefore all of the *place counters* are increased by 2. Furthermore, since the key was *fully matched*, RecenTo will increase the flow counter by 1.

In Fig. 3.3b packet p_2 of flow B is received. Note that flows A and B have somewhat similar keys. This similarity causes the key parts in some of the P-registers to match, while others do not match. Since each P-register is handled independently, some *place* counters will be increased and others decreased. As can be seen, the key in the first P-register does not match and thus its *place* counter will be decreased, while all of the others do match so their *place* counters will be increased. Since some parts of the key did not match, this is considered a *key mismatch*, and thus the *flow* counter will not be changed.

In Fig. 3.3c, packet p_3 of flow C is received. Here, only the last P-register matches and its *place* counter is increased, while *place* counters in the other 3 P-registers are decreased. The key was not matched and therefore the *flow* counter is not updated. Note however, that the *place* counter in the first P-register now equals 0, indicating that the P-register is essentially empty and a different key may be placed in this P-register. Note that since the key has not yet changed, if another packet of flow A arrives at this point, it can still update the counters and the F-register will still correctly represent the estimation of the flow's count.

In Fig. 3.3d, packet p_4 again of flow C is received. And now the first P-register is considered empty, so the new *part* of key of flow C is inserted. This illustrates the *key replaced* scenario. Since the flow key has been changed, the value of the *flow* counter is no longer relevant, and therefore it is reset to 0 so it may represent the count of the newly stored key (although the stored key is not valid, i.e. it is not a key that was seen in the stream thus far).

Finally, in Fig. 3.3e, packet p_5 of flow B is received. The first register is still considered empty (note that by design we don't update the *place* counter in the *replace key* case, only the *key*). and the new key of flow B is inserted. This illustrates the *key replaced* scenario. And again the F-register will be

reset and then incremented to 1 since after the key was replaced the key now fully matches. Note that the F-register will be reset every time any of the key parts in the P-registers are changed.

3.4 Handling Duplicate Keys

As described above, a new key will take the first empty place it finds, even if the same flow is already being counted in a later table, which could create duplicate counters for the same flow key. Since RecenTo does not want to maintain duplicate flow counts, it is necessary to remove one of the duplicate entries. Since updating the first instance will require recirculation, RecenTo will always delete the second instance, which will be done by setting the counter in the F-register to 0 (note that we can't update the P-registers, as this would also require recirculation). We point out that the counter values in the second instance could contain information which may potentially be lost.

Fix threshold. In order to handle the lost data of the second instance RecenTo defines the $fix_threshold$. If the information in the second instance is important enough, i.e., the *flow* counter is greater than the defined $fix_threshold$ value, RecenTo will recirculate the packet to update the first instance with the information that was deleted from the second instance. The recirculated packet will also clean the P-registers of the second instance. But if the *flow* counter does not cross the defined threshold, only the F-register will be reset, indicating that the key is no longer being counted, but the counters in the P-registers would need to gradually decrease before a new key can be inserted to that location. We discuss how the threshold be chosen in Section 6.1.2.

Example. In Fig. 3.4 we present an example of RecenTo processing with more than one table (here $d = 2$) and how duplicates are formed and handled. For ease of visualization, we assume the key is smaller, and can fit in 2 bytes, so *one* P-register is enough for the entire key and its corresponding place counter. The figure shows a single index in each one of the two tables, and both keys in the example map to these indexes. Note that one packet arrives in each row, and *inc* is set to 2.

We start the example with the tables as shown in the *initial state*. Packet p_1 of flow B arrives. Its key has a mismatch in Table 1, so only the *place* counter is decreased (and the *flow* counter is not changed). In Table 2 the key matches so both the *place* counter and the *flow* counters are increased

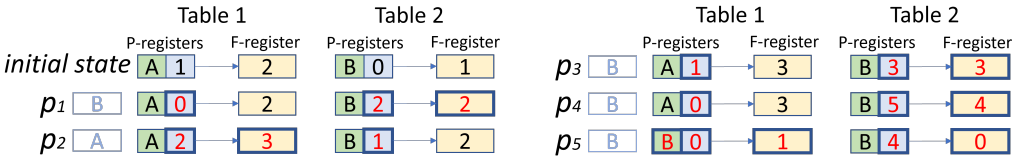
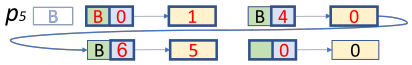
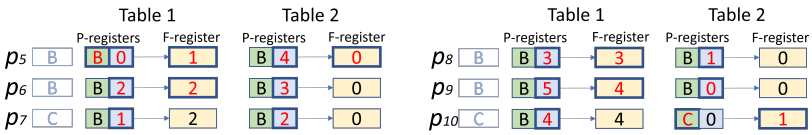


Figure 3.4: Formation of duplicate entries. $inc = 2$



(a) Recirculation due to duplicate keys: update first instance, clear the second one.



(b) Deleted F-register counter below threshold. Counters in P-register gradually decrease.

Figure 3.5: Handling duplication

(*place* counter increased by $inc = 2$, *flow* counter by 1). Then packet p_2 of flow A arrives. Its key matched in Table 1, so both counters are increased, and the key mismatched in Table 2, so the *place* counter is decreased. Then packets p_3 arrives followed by p_4. Both of these packets are of flow B, so for each the *place* counter in Table 1 is decreased and both *place* and *flow* counters in Table 2 are increased. At this point, packet p_5 of flow B arrives and finds an empty location in Table 1. Since it is not yet known that the flow already exists in Table 2, it will be inserted in Table 1, forming duplicate entries of flow B. Note that the *place* counter in Table 2 is decreased even though the key matches, since this key was already found in a prior table, and the *flow* counter is reset (if there were multiple P-registers we would need to check that the whole key matches, in which case the *flow* counter would be reset and incremented).

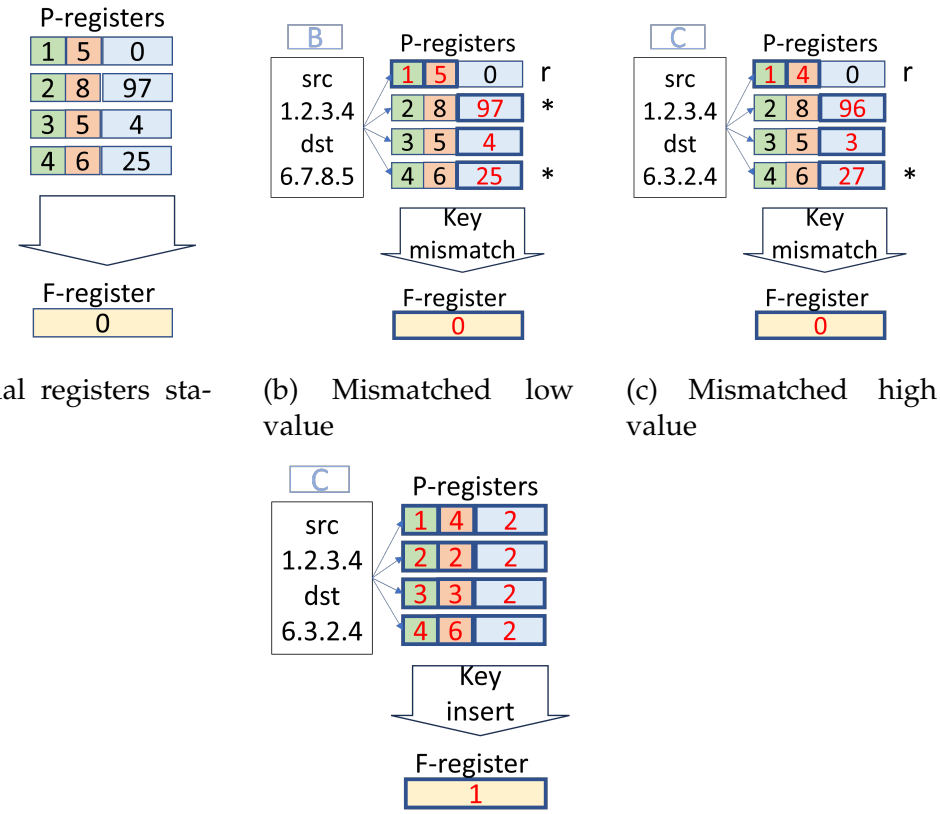
Once the duplicate is detected RecenTo will decide if the packet will be recirculated based on the $fix_threshold$ value. In Fig. 3.5 we can see both cases: In Fig. 3.5a, *fix_threshold* = 3 the deleted *flow* counter is above the threshold, so the packet will be recirculated. The counter values from the deleted instance will be added to the new entry, so the new counters will be: In the P-register: the old place counter$+inc$ (i.e., $4 + 2$) and in the F-

register: the old flow counter+1 (i.e., $4 + 1$) (Note that when the P-registers are inconsistent, we will update all the P-registers according to the smallest *place* counter found in that location). In addition, the P-register of the second instance will also be cleared. In Fig. 3.5b *fix_threshold* = 12, the threshold is higher than the counter in the deleted F-register, so RecenTo only resets the counter in the F-register. That is, the packet is not recirculated and so the P-registers of the second instance will not be reset. Now with the coming packets, the counter in the P-register will gradually decrease. As we can see packet p_6 of flow B matches in Table 1 and increases both counters. However, because the key was found in Table 1, it will decrease the counter of the P-register in Table 2 even though there is a match. Packet p_7 of flow C mismatches in both tables, so it decreases the counters in the P-registers in both tables. Packets p_8 and p_9 of flow B increase counters in Table 1, and decrease the counter in the P-register in Table 2. Finally, when p_{10} of flow C arrives, the counter in the P-register is zero, so C can can be inserted into this location in Table 2.

3.5 RecenTo improvements

Due to the fact that we maintain c independent P-registers and also a F-register, incompatibility can arise between them. There could be high variability between the P-registers in the same index, such that some are large while others are small. This could happen in the *key replace* scenario (e.g., Fig. 3.3d). There could also be a difference between the P-registers and the F-register, so that the F-register may be 0, while some of the P-registers still have a high count, which could be caused by a duplication (e.g., Fig. 3.5b). In these cases, if at least one of the P-register counters is large, this index will be *stuck* with an invalid key for a long time, preventing new flows from being counted.

Insert threshold. For these cases we define the *insert_threshold*. This threshold indicates what counter value of a P-register is considered too high, meaning the index will be *stuck* for a long time before it is reset. If both the F-register is set to 0 *and* if *any* of the P-registers are above the defined threshold, RecenTo will recirculate the packet in order to reset the counters in all the P-registers in this index. Otherwise, that is the counters in the P-registers are lower than the threshold, they will not be reset, and incoming packets will be processed until a new flow will manage to enter this index. We will expand further on how the insert threshold should be chosen in Section 6.1.2.



(a) Initial registers status

(b) Mismatched low value

(c) Mismatched high value

(d) Key inserted

Figure 3.6: insert case. $inc = 2$; $insert_threshold = 50$ (*) represents a partial key match; (r) indicates key has been replaced.

In Fig.3.6 we can see an example of using the insert option. In this example we use $inc = 2$ and $insert_threshold = 50$. Fig. 3.6a presents the incompatibility that can arrise between the counters. While some of the *place* counters are high, and some low, the *flow* counter's value is zero, thus indicating it is a free location. In Fig. 3.6b, a packet from flow B arrives, RecenTo finds the P-registers that need to be increased and decreases all other counters according to the *partial* key that is stored in each one of them. Since not all the *partial* keys are matched RecenTo will follow the *mismatch case*. But in the *flow* register RecenTo finds that this location is actually free. At this point the algorithm can't insert B into this location without a recirculation. As the recirculation requires more processing from the switch, RecenTo will not recirculate any packet that finds a free location, instead it will use the defined *insert_threshold* to decide if recirculation is needed. In this example the max *place* counter that was found is only 4 (we ignore all the *place* counter that had matched)- this means that only 4 packets are required in order to successfully insert this flow without recirculation. As the threshold is higher than this max *place* counter - it means that potentially loosing 4 packets would be admissible, so no recirculation will be made. Next we can see in Fig. 3.6c a packet from flow C arrives, now when RecenTo checks all of the P-registers, it finds that the maximum *place* counter of the mismatched key is 96. This value is higher than the threshold, which means that this location will be stuck for a long time before C will succeed to take this location, therefore recirculation will be made. You can see the inserted C flow in Fig. 3.6d.

Configuring the cleaning mechanism of the heavy hitters. As we describe in Section 3.3, the *inc* variable allows RecenTo to control the clearing mechanism. that is, *inc* will be set to a high value in order to avoid quick clearing of new items. In addition to preventing the rapid release of a new flow, we want to control the amount of time that old data is saved in the switch. While small flows will be evicted from the switch very quickly, a large stored flow will take longer to be evicted. We would like to have the option to limit the time such a large flow can remain in the switch after it has stopped being active. In order to control the timing of evicting old items from the switch, we can change the maximum value allowed in the *place counter* (e.g. in Alg. 1 line 2, the maximum value was set to $0xffff$) By restricting the counter size, all the heavy hitters that cross this value will get the same *place counter*, regardless of their *flow counter*, and will be evicted from the switch after the same period of inactivity.

3.6 The full algorithm

In Algorithm 2 we see the full RecenTo algorithm. In lines 1-20 we see the main loop, that will handle the inserted packets in the first pass over the switch in each of the d stages: lines 3-12 handle the packet up until the key is found in a stage, lines 4-6 split the key into c partial keys, and apply the update_key_reg function for any *partial* key, as described earlier in Alg. 1. Lines 7-9 update the flow counter in the *replace case*, it sets the flow counter to 0; if *all* the functions respond with 'match' (or 'replace'), in lines 10-12 the flow counter will be increased. Lines 13-19 handle packets of flows that were already found in a prior stage, line 14 will just decrease the place counter regardless of its key value. In lines 15-18 RecenTo checks for the *duplication case*. If all key parts report a 'match', a duplicate entry has been found. In this case, the full counter will be deleted from the second location and will be saved to decide on recirculation.

Lines 21-30 will be executed after the packet has been processed by all of the stages, there are still two cases that the algorithm needs to handle. First, in lines 21-25 the insert case, if a match was not found, but a free location was found, there is an option to recirculate the packet and insert it in the free location. The decision if recirculation will be made depends on the max place counter. If it crosses the *insert_threshold* (line 22) this location may be stuck for a long time, so recirculation will be made (line 23), otherwise the algorithm does nothing. Second, in lines 26-30 we see the fix case, if the flow is found twice, meaning the second match was a duplicate and it's full counter was deleted from the switch, there is also an option to recirculate the packet and fix the first instance with the deleted data. The decision to recirculate here depends on different a threshold, the *fix_threshold*. If the deleted data was important enough, i.e. greater than the threshold (line 27), the deleted information will be added to the first instance (line 28), otherwise the algorithm just ignores the deleted data.

At the end of the algorithm, lines 31-40, we can see how the recirculated packets are handled. If a 'fix' is needed (line 32-34), the first instance will be updated with the deleted information from the second instance (line 33), and the P-registers of the second instance will be cleared (line 34). If 'replace' is needed (line 35-37), the free location will be updated with the key of the recirculated packet (line 37), and all place counters will be reset to the initial value (line 36) (*inc* for place counter, and 1 for flow counter).

Algorithm 2 RecenTo - Full implementation

```
for i ← 1 to d do                                   #for any stage do the following
  l_i ← h_i(iKey)                                   #use hash function
  if This package didn't updated already then #split the iKey to c parts,
  and check independently each part
    for j ← 1 to c do
      Call update_pid_reg(l_i, j, iKey[j])
    end for
    if Key replaced case then
      Reset the F-register
    end if
    if Key fully matched case then
      Increase the F-register
    end if
  else                                  #match already made in previous table
    Independently decrease c place counters, for counter big than 0
    if All keys part had matched  then                  #duplication found
      Save the F-register in order to fix first instance
      Reset the F-register
    end if
  end if
end for
if Packet didn't update and one F-register is reset then        #there is an
empty location
  if At least one P-register cross the insert_threshold then
    Clone and recirculate package
  end if
end if
if Duplication found then        #second instance deleted and may be fixed
  if saved fix data cross fix_threshold then
    Clone and recirculate package
  end if
end if
if Packet is recirculated then                                  #a fix required
  if Case 1 - Fix needed then
    Update first instance with the saved counter (P and F registers)
    Delete P-registers of second instance #The F-register already cleared
  else if Case 2 - Replace needed then
    Update P-registers to initial value on the empty location
    Update F-register to initial value
  end if
  Drop the cloned copy
end if
```

4 Implementation

In this chapter we will present our code implementation of RecenTo in P4, We implemented p4 code for the Barefoot Tofino Wedge-100 programmable switch, using 500 lines of code [3].

4.1 Data structures

P4 code uses 2 different types of data structures, the first one will be saved in each stage memory, and will be available for the next packet that will traverse through the pipeline, this type of memory has very limited access and can be accessed only from the same stage. In addition, together with the packet, additional data can be maintained. This data can be used during the entire pipeline but will be lost when the packet will leave the pipeline.

Table structure. The tables will be saved in the stage memory, so all the proceeding packets can access this saved data. As we mentioned, due to the *mutual dependency*, the *place*-counter and the key should be handled in one P-register. In Fig. 4.1 we can see how we define the P-register. Note that the place-counter can also hold less then 16-bits, based on the max place counter the application wants to achieve.

Using this P-register struct, we can define the full table used by RecenTo, see in Fig 4.2 the declaration of the 4 P-registers tables and F-register table

```
struct p_register_pair_t {
    bit<16> key;
    bit<16> place_counter;
}
```

Figure 4.1: P-register definition

```
// == Register arrays for the stateful data structure

    Register<p_register_pair_t,_>(32w65536) p_register_1_1_R;
    Register<p_register_pair_t,_>(32w65536) p_register_1_2_R;
    Register<p_register_pair_t,_>(32w65536) p_register_1_3_R;
    Register<p_register_pair_t,_>(32w65536) p_register_1_4_R;
    Register<bit<32>,_>(32w65536) f_register_1_R;
    Register<p_register_pair_t,_>(32w65536) p_register_2_1_R;
    Register<p_register_pair_t,_>(32w65536) p_register_2_2_R;
    Register<p_register_pair_t,_>(32w65536) p_register_2_3_R;
    Register<p_register_pair_t,_>(32w65536) p_register_2_4_R;
    Register<bit<32>,_>(32w65536) f_register_2_R;
```

Figure 4.2: P-registers and F-registers definition

for d instances (Here $d = 2$). In this example each table contain 65,536 registers (0x10,000, totally up to 130K items can be tracked by the algorithm), Note that for evaluation purpose we used significantly less items per table.

Packet information structure. Together with the packet, important information is traversed in the pipeline, using this information RecenTo can make the decision for the given packet. See in Fig 4.3 the information we stored in this structure. Among others, we can see the 4 key parts that will be mapped to each of the respective P-register, the hash location for each d instance and the match indication that was reported by each P-register update.

```
struct ig_metadata_t {
    resubmit_data_64bit_t resubmit_data_read;
    resubmit_data_skimmed_t resubmit_data_write;

    //64bit Keys
    bit<16> key_part_1;
    bit<16> key_part_2;
    bit<16> key_part_3;
    bit<16> key_part_4;

    //register index to access (hash of key, with different hash functions)
    bit<16> stage_1_loc;
    bit<16> stage_2_loc;

    //is partial key matched register entry?
    bit<16> p_key_matched_1_1;
    bit<16> p_key_matched_1_2;
    bit<16> p_key_matched_1_3;
    bit<16> p_key_matched_1_4;
    bit<16> p_key_matched_2_1;
    bit<16> p_key_matched_2_2;
    bit<16> p_key_matched_2_3;
    bit<16> p_key_matched_2_4;

    bool matched_at_stage_1;
    bool matched_at_stage_2;

    bool replaced_at_stage_1;
    bool replaced_at_stage_2;

    //fix information
    bit<32> fix_flow_counter;
    bit<16> fix_place_counter;

    //Saved the flow counter
    bit<32> counter_read_1;
    bit<32> counter_read_2;
}
```

Figure 4.3: Packet information

```
void apply(inout p_register_pair_t value, out bit<16> rv) {
    p_register_pair_t in_value;
    in_value = value;
    if(in_value.key==ig_md.key_part_## pi ){
        /*Assume for this specification that counter value saturate
            at his maximum value rather than wrapping around.*/
        value.place_counter=in_value.place_counter |+| INC;
    } else {
        rv = in_value.place_counter;
        if(in_value.place_counter<=1){
            value.key=ig_md.key_part_## pi;
        } else {
            value.place_counter=in_value.place_counter-1;
        }
    }
}
```

Figure 4.4: P register match action in P4 code

4.2 P-registers update

Our biggest challenge was to implement the mutual dependency between the *key* and the *counter*. Since this is an operation that needs to be done on the same register - it cannot be too complex. We worked here to define the minimum number of operations needed for the algorithm. As you can see in Algorithm 1, in the 'match' case lines 1-7, the overflow check is handled by the language, and no special check needs to be done (using the $|+|$ operation). In the 'replaced' case lines 8-10 we defined the update to the partial key as mandatory, but updating the related place counter to *inc* value is not necessarily always needed. In fact, sometimes not updating this value will actually work better, as the tiny flows with just one packet, will never be able to occupy a place in the table, since at least two packets are needed for updating the place counter. On the other hand when the distribution is too high, or the amount of counters is too small, this field might need to be set as well. Finally, in the mismatch case in lines 11-13, the counter just needs to be decreased. The implementation of this algorithm in p4 can be found in Fig. 4.4.

In addition in Fig. 4.4 we can see how we return the status of the partial key (it is done by the *rv* out variable). As the rv can be set only once, but we need to report 3 different states, we used the default $rv = 0$ for the 'match' case, so no update needs to be done. In other cases we set the *rv*

```
// == Define Fully match check after all P_registers reported status ==
#define Table_set_match(pi)
    action set_matched_at_stage_## pi ##_(){ ig_md.matched_at_stage_ ## pi =true;}
    table tb_set_matched_at_stage_ ## pi ##_ {
        /*Fully match only if all P-registers report match*/
         actions = {
            set_matched_at_stage_## pi ##_;
            nop;
        }
        key = {
            ig_md.p_key_matched_## pi ##_1: exact;
            ig_md.p_key_matched_## pi ##_2: exact;
            ig_md.p_key_matched_## pi ##_3: exact;
            ig_md.p_key_matched_## pi ##_4: exact;
        }
        size = 16;
        default_action = nop;
        const entries = {
            (0, 0, 0, 0): set_matched_at_stage_## pi ##_();
        }
    }
```

Figure 4.5: Set match case indication

with the place counter, in this way we will get $rv = 1$ for the 'replace' case, and $rv > 1$ for the 'mismatch' case.

4.3 Check for fully match case

After performing the 4 independent checks on all parts of the key, it is necessary to check the full match of the key before updating the F-register. In Fig. 4.5 we can see how this verification is done. in the entries we define that only if all the key_matched reported 0- the fully match is set, otherwise no operation is done (see default_action set to nop.)

Similarly to the match action - we check for the replace case, See in Fig. 4.6. Here, unlike the match check, it is enough for one to report a replacement in order to receive a full replace, therefore in the entries we define 4 different cases that will cause a replace update. All other options will be ignored (again default_action set to nop.)

```
// == Define Replace case check after all P_registers reported status ==
#define Table_set_replace(pi)
    action set_replaced_at_stage_## pi ##_(){ ig_md.replaced_at_stage_ ## pi =true;}
    table tb_set_replaced_at_stage_ ## pi ##_ {
        /*Replace case if at least one P-registers report 1*/
         actions = {
            set_replaced_at_stage_## pi ##_;
            nop;
        }
        key = {
            ig_md.p_key_matched_## pi ##_1: ternary;
            ig_md.p_key_matched_## pi ##_2: ternary;
            ig_md.p_key_matched_## pi ##_3: ternary;
            ig_md.p_key_matched_## pi ##_4: ternary;
        }
        size = 16;
        default_action = nop;
        const entries = {
            (1, _, _, _): set_replaced_at_stage_## pi ##_();
            (_, 1, _, _): set_replaced_at_stage_## pi ##_();
            (_, _, 1, _): set_replaced_at_stage_## pi ##_();
            (_, _, _, 1): set_replaced_at_stage_## pi ##_();
        }
    }
```

Figure 4.6: Set replace case indication

```
table tb_update_f_register_1 {
    /* In Fully match- increase, In replace- write 1 and in resubmit Fix*/
    actions = {
        exec_stage_1_counter_incr;
        exec_stage_1_counter_write;
        exec_stage_1_counter_fix;
        nop;
    }
    key = {
        ig_intr_md.resubmit_flag: exact;
        ig_md.replaced_at_stage_1: ternary;
        ig_md.matched_at_stage_1: ternary;
    }
    size = 16;
    default_action = nop;
    const entries = {
        (0, true, _):      exec_stage_1_counter_write();
        (0, false, true): exec_stage_1_counter_incr();
        (1, _, _):         exec_stage_1_counter_fix();
    }
}
```

Figure 4.7: Update F-register

4.4 F-register update

Now, after the match and replace indications is updated based on the P-registers information, we can handle the F-register accordingly. See in Fig 4.7 the defined entries, if the packet is not resubmitted (in resubmit case only fix action is required) if the replaced flag is set, we need to write hard coded 1 in the F-register (as the old information is no longer relevant). However, if the replace is not set, and the match is set, the F-register is increased (for any other option the F-register is not updated).

For the second F-register table, the handling is little different, as in the second table duplicate key may be found, in this case the information in the second F-register will be deleted. See in Fig 4.8 the new entry, if both fully match indication is set - the write action will be selected. Only if no match is found in the first table, and no replace in the second table, the match in the second table will be handled by increasing the F-register. Note that in the second table no action is required in case of resubmit packet (the action is already done before the resubmit).

```
table tb_update_f_register_2 {
    /* In Fully match- increase, In replace- write 1 and in resubmit Fix*/
    actions = {
        exec_stage_2_counter_incr;
        exec_stage_2_counter_write;
        nop;
    }
    key = {
        ig_intr_md.resubmit_flag: exact;
        ig_md.replaced_at_stage_2: ternary;
        ig_md.matched_at_stage_1: ternary;
        ig_md.matched_at_stage_2: ternary;
    }
    size = 16;
    default_action = nop;
    const entries = {
        (0, true, _, _):          exec_stage_2_counter_write();
        (0, _, true, true):       exec_stage_2_counter_write();
        (0, false, false, true): exec_stage_2_counter_incr();
    }
}
```

Figure 4.8: Update F-register for second table

4.5 RecenTo logic

After we sett all the required decisions in the tables match action (using the table entries), the RecenTo logic is pretty simple, we just need to apply all the tables action for all the tables as shown in Fig 4.9.

```
// === Table 1 ===

// Apply all P-registers to check them status
_OAT(update_p_registers_1_1);
_OAT(update_p_registers_1_2);
_OAT(update_p_registers_1_3);
_OAT(update_p_registers_1_4);

// Set match only in case all the P-registers reported match
_OAT(set_matched_at_stage_1_);
// Set replace if at least one P-register reported replace
_OAT(set_replaced_at_stage_1_);

// update the f register - based on the matched and replaced result
_OAT(update_f_register_1);


// === Table 2 ===

 // Apply all P-registers to check them status
_OAT(update_p_registers_2_1);
_OAT(update_p_registers_2_2);
_OAT(update_p_registers_2_3);
_OAT(update_p_registers_2_4);

// Set match only in case all the P-registers reported match
_OAT(set_matched_at_stage_2_);
// Set replace if at least one P-register reported replace
_OAT(set_replaced_at_stage_2_);

// update the f register - based on the matched and replaced result
_OAT(update_f_register_2);
```

Figure 4.9: RecenTo logic

```
// === If fix needed, run recirculation (actually resubmit) ===
bool is_resubmitted=(bool) ig_intr_md.resubmit_flag;
if(!is_resubmitted && ig_md.matched_at_stage_1 && ig_md.matched_at_stage_2){
    //none matched
    //prepare for resubmit!
    if(ig_md.resubmit_data_write.fix_counter & THRESHOLD_FIX != 0){
        clone_and_recirc_replace_entry();
    }
}
```

Figure 4.10: Resubmit decision based on the fix threshold

4.6 Recirculation decision

In Fig. 4.10 we can see the end of the pipeline, here we have the information where the item was found, in case it is found in both tables, a fix is required. In order to reduce recirculation amount - we use the fix threshold, so not each fix will require recirculation (if the packet will not recirculate, the data that was deleted will be lost).

In order to optimize the code, the threshold values in our implementation can't get any number as threshold value, but only a threshold of $2^c - 1$ (i.e. sequence of a 0's, following by b 1's: 0^a1^b e.g. $0b0000111$). For such threshold we can use the & operator instead of $>$. That is, we can easily check if the *and* operator of the max counter with the *not* operator of the threshold is not 0. If so the counter is higher than the threshold, e.g $max_counter \& 0b1111000! = 0$).

Note, as the *insert_threshold* requires many resources, since RecenTo needs to check the *threshold* for each partial key d times). As we will see in section 6.1.2, this threshold has a minor effect on the accuracy, thus we removed this part of code from our implementation.

4.7 RecenTo resources

Our P4 implementation tracks a total of $128K$ flows, divided into two tables ($d = 2$) with $64k$ flows per table. Our implementation used no more than 30% of any of the switch resources. Specifically, the implementation used 25% of the available SRAM, less than 20% of the available Packet-Header-Vector (PHV) and 2 hash functions (since $d = 2$). See in Fig. 4.11 the used resources of our implementation as provided by p4i tool.

Resource Summary | File... | Pipe 0, RecenTo.p4

Show fractions

	Average	0	1	2	3	4	5	6	7	8	9	10	11
Action Data Bus Bytes	2.3%	0.0%	4.7%	3.1%	3.1%	0.0%	3.9%	3.1%	3.1%	3.1%	0.0%	3.1%	0.0%
8-bit Action Slots	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
16-bit Action Slots	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
32-bit Action Slots	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Exact Match Input Xbar	4.2%	0.0%	6.3%	5.5%	5.5%	6.3%	3.1%	3.1%	4.7%	10.9%	0.0%	2.3%	2.3%
Gateway	3.1%	0.0%	12.5%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	6.3%	18.8%
Hash Bit	5.3%	0.0%	7.7%	8.7%	8.7%	9.6%	3.8%	3.8%	3.8%	13.5%	0.0%	3.8%	0.0%
Hash Dist Unit	12.5%	0.0%	33.3%	16.7%	16.7%	0.0%	16.7%	16.7%	16.7%	16.7%	0.0%	16.7%	0.0%
Logical Table ID	12.0%	18.8%	12.5%	12.5%	12.5%	6.3%	6.3%	6.3%	18.8%	12.5%	6.3%	12.5%	18.8%
Map RAM	29.5%	0.0%	0.0%	70.8%	70.8%	0.0%	35.4%	35.4%	70.8%	35.4%	0.0%	35.4%	0.0%
Meter ALU	20.8%	0.0%	0.0%	50.0%	50.0%	0.0%	25.0%	25.0%	50.0%	25.0%	0.0%	25.0%	0.0%
SRAM	19.8%	0.0%	0.0%	45.0%	45.0%	5.0%	22.5%	22.5%	46.3%	27.5%	1.3%	22.5%	0.0%
Stash	3.1%	0.0%	0.0%	12.5%	12.5%	6.3%	0.0%	0.0%	0.0%	6.3%	0.0%	0.0%	0.0%
Stats ALU	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
TCAM	3.5%	0.0%	0.0%	0.0%	0.0%	0.0%	4.2%	4.2%	16.7%	4.2%	8.3%	4.2%	0.0%
Ternary Match Input Xbar	2.7%	0.0%	0.0%	0.0%	0.0%	0.0%	1.5%	1.5%	13.6%	1.5%	12.1%	1.5%	0.0%
VLIW Instruction	7.0%	6.3%	6.3%	9.4%	9.4%	3.1%	6.3%	6.3%	12.5%	9.4%	3.1%	6.3%	6.3%
Exact Match Search Bus	5.7%	0.0%	6.3%	12.5%	12.5%	6.3%	0.0%	0.0%	0.0%	6.3%	0.0%	6.3%	18.8%
Exact Match Result Bus	4.2%	0.0%	12.5%	12.5%	12.5%	6.3%	0.0%	0.0%	0.0%	6.3%	0.0%	0.0%	0.0%
Ternary Result Bus	7.3%	18.8%	0.0%	0.0%	0.0%	0.0%	6.3%	6.3%	18.8%	6.3%	6.3%	12.5%	12.5%

Figure 4.11: Resources summary as provided by P4i

5 Emulation Implementation

For evaluation purposes, to facilitate the examination of the various parameters used in the RecenTo algorithm, we implemented in addition to the p4 code, a simulator in python. This python code contains all the following parts:

RecenTo implementation. We implement the RecenTo in python, to simulate the dedicated memory per stage, we implement each stage with its memory in a different class instance, when all the stages are connected as a linked list. The P-registers as well are implemented as a class, so we can handle both *p_counter* and *key* in one action.

In addition to the RecenTo implementation, we implement all the other algorithms we wanted to compare our algorithm to:

PRECISION implementation. Similar to RecenTo , PRECISION also simulates standalone stages, each with its dedicated memory. While in RecenTo the dependence between the counters are much more influential, so we also split the F-counter stage from the P-counters stage, in PRECISION the code is much simpler, and both key and counter are handled in the same class instance. The last stage in the linked list are connected to the first stage, so PRECISION can run the useful recirculation action easily.

Heavy-Kipper implementation. HK can't run on a programmable switch, so instead of the linked list, we implement a simpler table. In addition to this table we implement the min_heap that can't be implemented in the switch design. The algorithm first accesses the min_heap, and after that accesses all the 'stages' in the table, and after that updates again the min_heap with the result.

Count-Min-Sketch implementation. The Count-Min-Sketch is handled a little differently, as a sketch based algorithm it stores only the counter information and not the key. Similarly to the real implementation, in each stage we create a table of the counters, so in each stage the packet can be hashed to several columns, but after some evaluation we noticed that

using one column per table provides better result, so we take this layout of the memory.

Dsketch implementation. Dsketch is very similar to the CMS, but it also implements the AED action to decrease the counters. In principle in HW the decrement depends on the timing, but it was a little hard to simulate it in python, as in the simulation the timing gives a slightly different result. Therefore, instead of taking the same timing as presented in the p4 code example of Dsketch, we check for the best timing result and take the evaluation with these parameters.

Testing information implementation. In all the algorithms above, we also implemented more functionality for testing usage, for example- count the recirculation amount that happened during the test, get the Top-K elements stored in the switch, and so on. This data was much more easier to collect in python emulation then in the P4 code on real HW.

6 Evaluation

Our evaluations explore two main aspects, the first is RecenTo parameter configuration and the second is a comparison of RecenTo to existing state of the art algorithms. In all of our tests we use the following metrics:

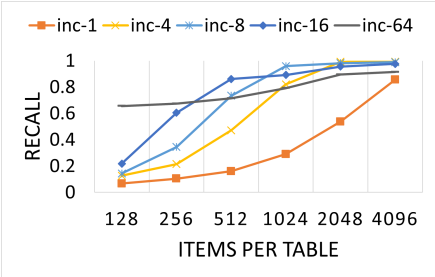
Average Relative Error (ARE). used to assess the accuracy of the *Frequency Estimation* of the flows, calculated by the difference between the real top-k counters, and their corresponding estimation on the switch. ARE is defined as:

$$ARE(Alg) = \frac{1}{k}\sum_{t=1}^{k}\frac{|Real - Estimation|}{Real}$$

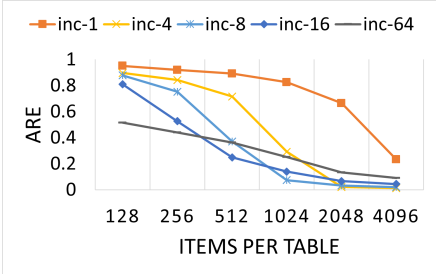
Recall. used to assess an algorithm's ability to report the *Top-K* flows. Recall is calculated by comparing the true-positive elements that are reported by the algorithm (out of a total of m reported elements), divided by the relevant elements (i.e., the true k heaviest flows), where $m >= k$. Formally, the recall metric is defined as:

$$Recall(Alg) = \frac{TruePositive}{RelevantElements}$$

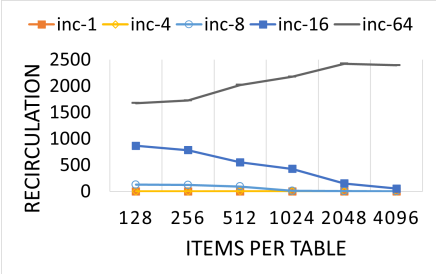
Parameters and data. In our tests we used $m = 2k = 256$ (compare top 256 elements counted by the algorithm with the top 128 element from expected result). Additionally, in all our tests, unless otherwise noted, we use a set of 5 traces from the CAIDA database[1], the results shown are an average across all 5 traces. For the sake of efficiency we will evaluate RecenTo on the first 2M packets from each trace. These traces hold around 1M unique flows. The flow count of the top-k flows is between 300 and 22K packets per flow. Additionally, we use $d = 2$. Using $d = 3$ provided diminishing returns, yet using this available memory to enlarge each of the two tables significantly improved the result, thus we opted for using $d = 2$. but the number of items x per table is defined separately for each test. We note that the total amount of registers required by RecenTo $5 * x * d$.



(a) Recall



(b) ARE



(c) Recirculation amount

Figure 6.1: Effect of number of counters per stage on the metrics, on first 2M packets of CAIDA traffic

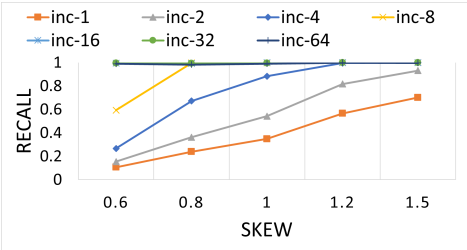
6.1 RecenTo accuracy

In this section we will evaluate the accuracy of RecenTo, and how it is affected by the different parameters of the algorithm including *inc*, *fix_threshold* and *insert_threshold*.

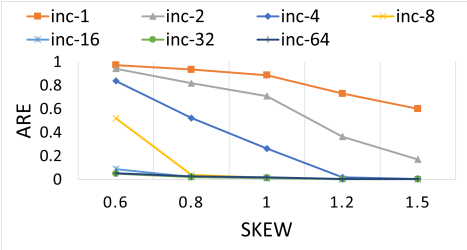
6.1.1 Choosing The *inc* Parameter

Recall that *inc* is the value by which *place* counters are incremented if the key is matched. As we described in section 3.3, *inc* should make up for the excess reductions that are made to the *place* counters when the key is *not* matched. Notice that if *inc* is too small, the *place* counters may be reset too quickly, and the algorithm may inadvertently remove heavy flows. On the other hand, if *inc* is too big, RecenTo won't be able to evict older flows, or small flows that are still in the table. Additionally, the choice of the *inc* value should be influenced by both the number of items maintained by the tables and the distribution or the skew of the traffic.

Varying the size of the table. Here we present the significant interaction



(a) Recall for different skew parameter



(b) ARE for different skew parameter

Figure 6.2: Effect of skew on the metrics, for 2M packets

between the number of items x per table and the *inc* value. In this test we evaluate a table size between 128 and 4096 items per table, and set *inc* to be in the range of 1-64. Figures 6.1a and 6.1b show the result of our test. When the number of items is small, a higher value of *inc* will generally provide higher accuracy. Let's take as an example the graphs of $inc = 8$ (inc-8) and $inc = 64$ (inc-64), we can see that for a table with 128 - 256 items an *inc* value of 64 does better across both metrics. For a table with 512 items we get similar results for both values of *inc*, but for 1024 items and higher an *inc* value of 8 does better than a value of 64. The reason for this is that with fewer items more packets are mapped to each register. Therefore the place counters are decreased more frequently causing them to quickly reset. In order for the place counters to be reset gradually a large value of *inc* is needed. On the other hand, if the table has more items we see that RecenTo achieves a slightly lower accuracy for large values of *inc*. This is because the place counters take longer to decrease (due to the fact that there are less collisions) therefore it takes time for new flows to enter the table, thus reducing their flow counter accuracy.

The effect on the amount of required recirculation is shown in Fig. 6.1c. Here we can see that higher values of *inc* results in more recirculations. The reason for this is that with higher values, place counters may be higher, causing more items to cross the *insert_threshold*, requiring more recirculation to be made. For example for an *inc* value of 64 (inc-64), the place counters will generally be larger than they would be with smaller values of *inc* (as each collision significantly increases the counter), thus the counters will pass the *insert_threshold* more often.

Effect of the *place counter* size on different λ over time. We test how the size of the *place counter* affects the tables over time. In Fig. 6.3 we can see when using a place counter of 1 byte - the maximum will be $0xff$,

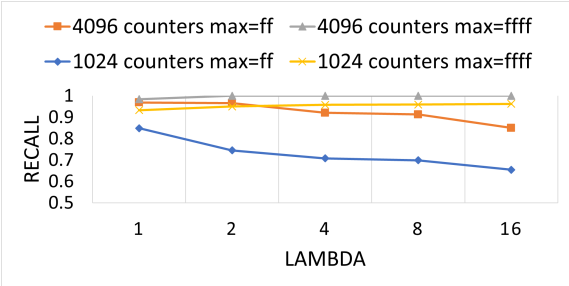


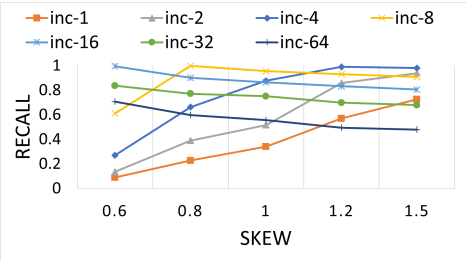
Figure 6.3: Effect of number of counters and maximum place counter on the λ-recent-frequency λ=(1M, 2M, 4M, 8M, 16M). After 16M CAIDA packets

and 2 bytes will provide a maximum of $0xffff$. We check RecenTo over time for various λ, using 1024 counters with $inc = 8$, and 4096 counters with $inc = 4$. We can see that for RecenTo to remember flows for a long time (here $\lambda = 16$) a higher max *place counter* needs to be used to enable RecenTo to save somewhat inactive flows. The amount of counters used also affects the result over time, since more counters provides RecenTo with more room to save old data, so that with the same max *place counter* the eviction will occur later.

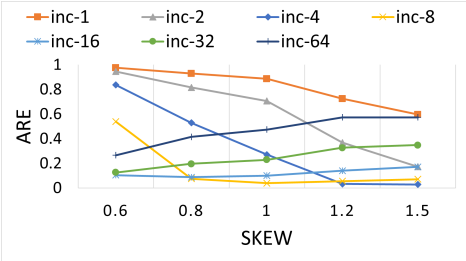
Note that in this test we used continuously 16M packets, therefore the accuracy over time is very high, and using high *place counter* help to save all the data to the end of the evaluation, but as we will see later, when the data over the switch being changed, small *place counter* will help to adapt faster to the new information.

Effect of traffic distribution. The distribution of the traffic may also affect the correct choice of *inc*. If the distribution is more skewed, the heavy items are more substantial in the stream and therefore relatively fewer items will affect the place counters of the heavy hitters. Fig. 6.2 shows the interaction between the traffic skew and the value of *inc*. We note that the value of *inc* can easily be changed on-the-fly, as the distribution changes, but our evaluations test only one value of *inc* each time. For this test we created a set of traces with a zipfian distribution, and a skew parameter of values in the range of 0.6 - 1.5 (the skew is computed as the Fisher-Pearson coefficient of skewness [4]). Each trace consists of $2M$ packets, and the tests shown use a table size of 1024 items.

Figures 6.2a and 6.2b show the results for the tested skew parameter. Here we can see that a lower skew of distribution (i.e., smaller zipf parameter) will require a larger value of *inc* to get better accuracy. Intuitively, the



(a) Recall second bulk



(b) ARE second bulk

Figure 6.4: Effect of skew on the metrics, for second bulk of 2M packets

reason for this is that with higher skew there are fewer heavy hitters and the rest of the flows are generally smaller. With a smaller skew there are more flows that are almost heavy. Collisions between heavy flows and almost heavy flows could cause the heavy hitters to be removed from the structure and even if the heavy flows eventually return to the structure their flow counter would be less accurate.

The affect of traffic distribution on the clearing mechanism. In Fig. 6.2 we used synthetic traffic that is evenly distributed, therefore the heavy hitters will mostly appear at the beginning of the trace. Therefore RecenTo does not need to make room in the table for new flows, so the high value of *inc* does not lower the accuracy and recall as seen in Fig. 6.1.

In order to also check the evicting mechanism in Zipfian traffic, we create another bulk of 2M packets, and send them to the switch after finishing with the first bulk that was shown in Fig. 6.2. The result can be seen in Fig. 6.4. We can see that a higher *skew* will require a smaller *inc* value to get better accuracy. The reason for this is that with higher *inc* old flows don't evict from the switch quickly enough, so important data is lost.

Note that this test is done with a *place* counter of size 2 bytes, so a high *inc* leads to a very high *place* counter, and makes it harder to remove old heavy hitters. When we do the same with only 1 byte allocated for the *place* counter, the second bulk gives similar result to those received in the first bulk, since old heavy hitters didn't stay in the switch for a long time, and the heavy flows from the second bulk succeeded to enter the switch more quickly.

Threshold	0	80	256	512	1024	4,096	MAX
Recall	0.73	0.72	0.76	0.73	0.69	0.57	0.21
ARE	0.36	0.37	0.36	0.4	0.47	0.58	0.84
Recirculation	50K	3K	3K	3K	2K	1K	100

Table 6.1: *Insert threshold* for 128 counters per stage

6.1.2 Setting the Thresholds to Limit Recirculation

The thresholds used by RecenTo determine when recirculation is needed. These thresholds need to be set high enough so that recirculation will not reduce throughput, yet if they are too high important information may be lost. RecenTo uses two different thresholds, the *insert_threshold* and the $fix_threshold$.

Insert-threshold. The *insert_threshold* defined in Section 3.5 determines when a recirculation is needed after a key was inserted into the table. We evaluate the affect of the *insert_threshold* on the accuracy of the algorithm. We run RecenTo using a table size of 128 items and $inc = 64$ accordingly and present the results for different metrics in Table 6.1. Generally, as the threshold is increased the number of recirculations decrease (note that a threshold of 0 means that recirculation is done every time a flow counter that is 0 is found, and and MAX means we recirculate only when at least one associated P-register counter reached the maximum value). We can see that the sweet-spot value for the *insert_threshold* is between 256 and 1024. Lower values result in a very high number of recirculations, and higher values get significantly lower recall and accuracy. Note that we also performed this test for a table with 1024 items. In this case the difference between the different thresholds was minor, since there are less collisions between keys causing less discrepancies between the place counters in each index.

Fix-threshold. The $fix_threshold$ defined in Section 3.4, determines when a recirculation is needed in case a duplicate key is found. We run RecenTo using a table size of 1024 items and $inc = 8$ accordingly and present the results for different metrics in Table 6.2. We examine a range of values for the threshold (a threshold value of 0 means that RecenTo recirculates any time a key was deleted due to duplication). As can be seen, the accuracy changes across the different threshold values was minimal, but the difference between the amount of recirculation was significant, indicating that a high $fix_threshold$ would be sufficient in this case. Note that this test was done using an initially empty table. We believe that over time

Threshold	0	32	128	256	2048	4096
Recall	0.98	0.98	0.97	0.97	0.96	0.96
ARE	0.04	0.05	0.06	0.07	0.07	0.07
Recirculation	800	60	18	11	9	9

Table 6.2: *Fix threshold* for 1024 counters per stage

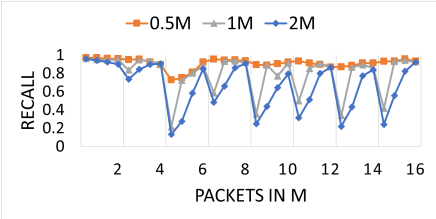
the affect of the $fix_threshold$ becomes more significant. In any case, it requires significantly less recirculations than was needed for the comparable $insert_threshold$.

6.2 Finding the Recently Heavy Flows

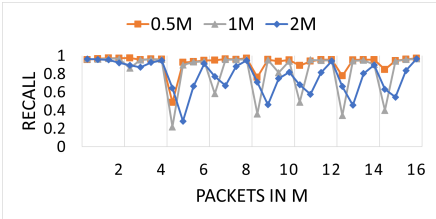
Our tests look at the ability to find heavy flows over time. To do so, we take $16M$ packets from a CAIDA trace, and divide it into 8 consecutive bulks of $2M$ packets each. In order to increase the turnover of flows and limit the length of each flow, we limit each flow so it is only valid for at most two consecutive bulks. For each flow, we randomly determine if it will be active for one or two bulks. Once a flow is no longer active, packets from that flow will be mapped to another fixed key, which will also be active for 1-2 bulks and so on. We test the accuracy and recall rate after each 0.5M packets (1/4 bulk) with different GTs, using λ=(0.5M, 1M, 2M) RecenTo runs continuously across all bulks, and the tables are not cleared between each bulk. Note that in order to adapt to the size of the bulks and show the eviction mechanism we set a maximum of 0xff for the *place* counters.

6.2.1 Recall for different λ values.

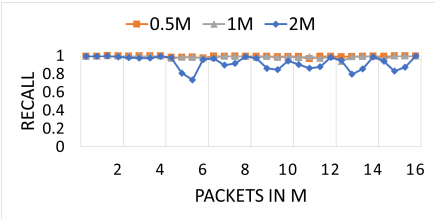
In Fig. 6.5 we show the recall rate of RecenTo with different parameters. Fig.6.5a presents the result when we used RecenTo with 1024 counters and inc=8. When $\lambda = 0.5M$, the recall rate is very high. We do get a small decrease in the beginning of each bulk, but RecenTo quickly replaces the data. The large change of flows between each bulk in our trace is also the reason for the decrease at the beginning of each bulk for $\lambda = 1M$ and $\lambda = 2M$. Here the GT also accounts for data from the prior bulk that were quickly cleaned out by RecenTo . In Fig.6.5b, we also 1024 counters, but with $inc = 16$. In this case, the recovery took longer. As we can see some results for $\lambda = 0.5M$ were lower, but the old data was saved by



(a) 1,024 items with inc=8



(b) 1,024 items with inc=16



(c) 4,096 items with inc=16

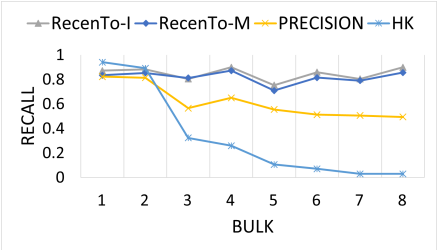
Figure 6.5: Recall with different λ λ=(0.5M, 1M, 2M). Note that in the trace many flows change after 2M packets.

RecenTo for longer, and therefore in $\lambda = 1M$ and $\lambda = 2M$ we get better results. In Fig.6.5c we use 4,096 counters, and $inc = 16$, here the result is better for all values of λ, since RecenTo has enough room to store more flows, and therefore more "old" flows still remain in the structure while new flows quickly succeeded to enter the structure. Note that since flow count accuracy is measured for the entire duration of the flow, it is not affected by the size of λ. We will discuss ARE rates later when comparing to different solutions.

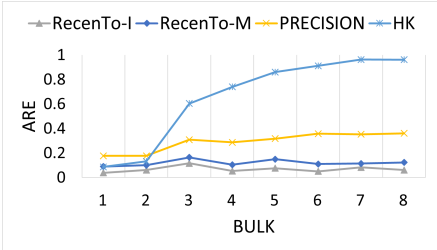
6.2.2 RecenTo vs existing algorithms.

We compare RecenTo's ability to identify and count the flows that were *recently* heavy to several state-of-the-art solutions, including PRECISION [7], HeavyKeeper(HK) [18], Dsketch [19] and CMS [14]. Tests were performed on the same data we create for prior tests using $\lambda = 2M$.

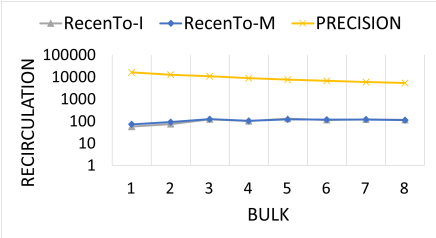
Comparing RecenTo to existing counter-based algorithms. We run the test using **PRECISION** with a table size of 1024 items and **HK** with a table size of 1536 items (which uses the same amount of memory). Recall that due to the use of multiple P-registers, RecenTo items require more memory. We therefore compare these to two different configurations of RecenTo. The first is **RecenTo-I** which uses the same number of *items* as



(a) Recall over time



(b) ARE over time



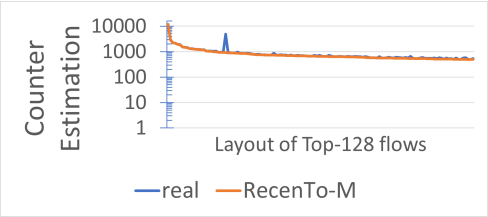
(c) Recirculation over time

Figure 6.6: 2M packet of random values per bulk, based on CAIDA traffic

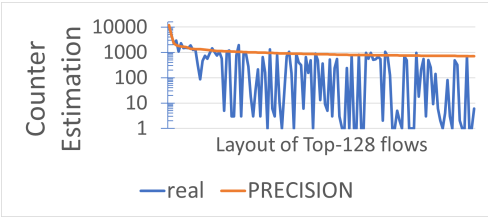
PRECISION, that is, it maintains 1024 items and uses $inc = 8$. The second is **RecenTo-M** which uses the same amount of *memory* as used by PRECISION and HK, that is, it maintains 614 items and uses $inc = 16$ accordingly.

In Fig. 6.6 we show the result after each bulk. Fig. 6.6a presents the **Recall** rate of the algorithms. As can be seen, in the first bulk the recall rate is similar for all algorithms, but after some time, RecenTo does significantly better since it allows new flows to enter the structure more easily. Notice that HK maintains the top-k flows in the min-heap and there is no process which cleans up the heap. Over time, we see that the new flows didn't succeed to enter the structure Fig. 6.6b presents the **ARE** metric. As shown, over time the error rate is lower for both RecenTo instances, for the same reason. Furthermore, Fig. 6.6c shows the number of **Recirculations** required by the algorithms. Both RecenTo instances used a relatively small number of recirculations, while PRECISION required a higher number of recirculations. Note that HK is not a data-plane algorithm, thus while we could compare the accuracy metrics, the recirculation information is not relevant. We can see here that as time passes PRECISION requires less recirculations. The reason is that as the counters get higher values, the probability to insert a flow and thus recirculate the packet decreases.

In Fig. 6.7 we can see the top-128 counter estimation that was reported



(a) RecenTo-M counter estimation always maintains lower bound of flow count.



(b) PRECISION counter estimation usually maintains upper bound of flow count.

Figure 6.7: Layout of 128 top flow detected by each algorithm in the last bulk

by each of the algorithms for the last bulk (ignoring items from previous bulks), compared to the real counts of these detected flows. This provides some indication of why the accuracy of PRECISION is reduced in later bulks. In Fig. 6.7a we see the result for the flows that had been detected by RecenTo-M (RecenTo-I gives slightly better results), it easy to see that RecenTo maintains a lower bound for the real counters. As a result, RecenTo won't report a flow as heavy when it really isn't (i.e., a false positive). On the other hand, PRECISION usually maintains an upper bound of the counts. As we can see in Fig. 6.7b, many of the flows that PRECISION reports as heavy actually have very low counts. We note that HK also maintains a lower bound for the real count of the flows. However as shown in Figure 6.6a the recall rate achieved by HK in the last bulk is very low (in fact only 4 of the top-k flows were identified in this bulk), yet the estimate HK provided for these flows was very close to the ground truth.

Comparing RecenTo to existing sketch-based algorithms. We compare RecenTo to two sketch-based algorithms, namely CMS[14] and Dsketch [19]. Since these are sketch-based algorithms, they do not maintain the flow keys and therefore can not be used in stand-alone to detect the Top-K flows. Therefore, we only compare the result for the ARE metric We run the same test as described before when using a CMS with 2 rows and 3072 items (i.e. registers) per row (same amount of memory as used for RecenTo-M), and Dsketch with 2 rows and 1536 items per row (since it also stores a time register per item thus requiring more memory for each item).

In Fig. 6.8 we present the ARE result for these algorithms. As can be seen, even in the first bulk, CMS has an ARE of 1, which means that for most of the counters CMS provided a counter estimation that is two times the

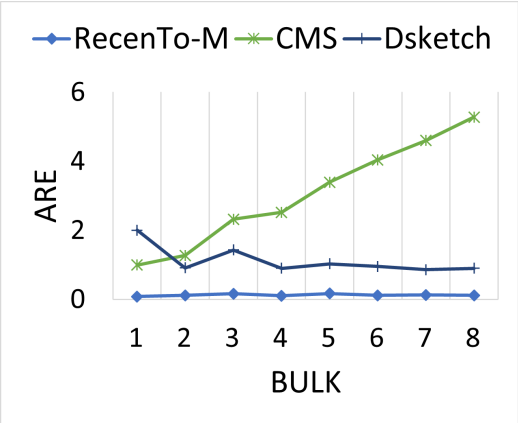


Figure 6.8: *ARE* for RecenTo-M vs. CMS and Dsketch.

GT. RecenTo remains with an ARE close to 0 across all bulks, meaning that the error is relatively small. When we check the last bulk, the error is even more significant, CMS provided a counter estimate of around 6 times the real counters. This makes sense, since the counters in CMS keep increasing all the time, thus the error keeps increasing. On the other hand, the ARE shown by RecenTo slightly increases between the first and last bulks, yet it is still relatively low, thanks to RecenTo's inherent key eviction process. Notice that Dsketch does better than CMS yet slightly worse than RecenTo. The Dsketch result also stays similar for all bulks, as in RecenTo, since Dsketch has an incorporated decay process. That said, Dsketch required around 1000 recirculations per 2M packets while RecenTo only required around 100 recirculations, and CMS did not use recirculations at all.

7 Conclusion

In this work we presented the importance of finding the heavy hitter problem in programmable switches. In addition, we present the *recent top-k* problem and the importance of referring to recently updated information, and especially the recent heavy hitters. To the best of our knowledge, many of the existing algorithms present results in a continuous manner without reference to time. Although there are solutions which can perform heavy hitters detection in relation to time, most of them make use of intervals, and require additional resources from the system, which makes it difficult to realize them in the data plane. In order to provide a unified solution to this problem, without requiring additional resources, we present RecenTo, a counter-based algorithm for finding the *recent* heavy flows in the data plane. RecenTo does not require a separate process for defining data as recent. We show our ability to control the process of evacuating information from the switch, when the time to evacuate new and small items is controlled by the *inc* value, and the time to evacuate heavy and old items is controlled by the maximum value of the *place counter* that we use. We show that RecenTo provides high accuracy for both the flow count estimation and in finding the recent top-k items, for both Zipfian synthetic traffic, and real-world CAIDA traffic using limited resources and minimal recirculations. We show RecenTo maintains high accuracy over time for recent heavy hitters, thanks to the RecenTo eviction scheme. In future work we plane to examine how RecenTo may be tailored to additional network hardware architecture as well as how additional metrics can be adapted to measure the recent past.

Bibliography

[1] Caida datase. https://publicdata.caida.org/datasets/security/, 2018. [Online; accessed December 2023].

[2] P4 tutorial sigcom 2019. https://github.com/p4lang/tutorials/, 2019.

[3] Recento p4 code. https://github.com/aviacoh/RecenTo$_p4_code$, 2023. [*Online*; *accessedDec*

[4] scipy.stats.skew. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.skew 2023. [Online; accessed December 2023].

[5] AFEK, Y., BREMLER-BARR, A., FEIBISH, S. L., AND SCHIFF, L. Detecting heavy flows in the SDN match and action model. *Comput. Networks 136* (2018), 1–12.

[6] BASAT, R. B., CHEN, X., EINZIGER, G., FRIEDMAN, R., AND KASSNER, Y. Randomized admission policy for efficient top-k, frequency, and volume estimation. *IEEE/ACM Trans. Netw. 27*, 4 (2019), 1432–1445.

[7] BASAT, R. B., CHEN, X., EINZIGER, G., AND ROTTENSTREICH, O. Designing heavy-hitter detection algorithms for programmable switches. *IEEE/ACM Trans. Netw. 28*, 3 (2020), 1172–1185.

[8] BOSE, P., KRANAKIS, E., MORIN, P., AND TANG, Y. Bounds for frequency estimation of packet streams. 33–42.

[9] BOSSHART, P., DALY, D., GIBB, G., IZZARD, M., MCKEOWN, N., REXFORD, J., SCHLESINGER, C., TALAYCO, D., VAHDAT, A., VARGHESE, G., AND WALKER, D. P4: programming protocol-independent packet processors. *Comput. Commun. Rev. 44*, 3 (2014), 87–95.

[10] CHARIKAR, M., CHEN, K. C., AND FARACH-COLTON, M. Finding frequent items in data streams. *Theor. Comput. Sci. 312*, 1 (2004), 3–15.

[11] CHEN, X., FEIBISH, S. L., KORAL, Y., REXFORD, J., AND ROTTENSTREICH, O. Catching the microburst culprits with snappy. In *Proceedings of the Afternoon Workshop on Self-Driving Networks, SelfDN@SIGCOMM 2018, Budapest, Hungary, August 24, 2018* (2018), ACM, pp. 22–28.

[12] CHEN, X., FEIBISH, S. L., KORAL, Y., REXFORD, J., ROTTENSTREICH, O., MONETTI, S. A., AND WANG, T. Fine-grained queue measurement in the data plane. In *Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies, CoNEXT 2019, Orlando, FL, USA, December 09-12, 2019* (2019), A. Mohaisen and Z. Zhang, Eds., ACM, pp. 15–29.

[13] CORMODE, G. Misra-gries summaries. In *Encyclopedia of Algorithms*. 2016, pp. 1334–1337.

[14] CORMODE, G., AND MUTHUKRISHNAN, S. An improved data stream summary: the count-min sketch and its applications. *J. Algorithms 55*, 1 (2005), 58–75.

[15] ESTAN, C., AND VARGHESE, G. New directions in traffic measurement and accounting. *Comput. Commun. Rev. 32*, 1 (2002), 75.

[16] ESTAN, C., AND VARGHESE, G. New directions in traffic measurement and accounting: Focusing on the elephants, ignoring the mice. *ACM Trans. Comput. Syst. 21*, 3 (2003), 270–313.

[17] FEIBISH, S. L., LIU, Z., AND REXFORD, J. Compact data structures for network telemetry. *CoRR abs/2311.02636* (2023).

[18] GONG, J., YANG, T., ZHANG, H., LI, H., UHLIG, S., CHEN, S., UDEN, L., AND LI, X. Heavykeeper: An accurate algorithm for finding top-k elephant flows. In *2018 USENIX Annual Technical Conference, USENIX ATC 2018, Boston, MA, USA, July 11-13, 2018* (2018), H. S. Gunawi and B. C. Reed, Eds., USENIX Association, pp. 909–921.

[19] KHOOI, X. Z., CSIKOR, L., LI, J., KANG, M. S., AND DIVAKARAN, D. M. Revisiting heavy-hitter detection on commodity programmable switches. In *7th IEEE International Conference on Network Softwarization, NetSoft 2021, Tokyo, Japan, June 28 - July 2, 2021* (2021), K. Shiomoto, Y. Kim, C. E. Rothenberg, B. Martini, E. Oki, B. Choi, N. Kamiyama, and S. Secci, Eds., IEEE, pp. 79–87.

[20] LIU, Z., BEN-BASAT, R., EINZIGER, G., KASSNER, Y., BRAVERMAN, V., FRIEDMAN, R., AND SEKAR, V. Nitrosketch: robust and general sketch-based monitoring in software switches. In *Proceedings of the ACM Special Interest Group on Data Communication, SIGCOMM 2019, Beijing, China, August 19-23, 2019* (2019), J. Wu and W. Hall, Eds., ACM, pp. 334–350.

[21] LIU, Z., NAMKUNG, H., NIKOLAIDIS, G., LEE, J., KIM, C., JIN, X., BRAVERMAN, V., YU, M., AND SEKAR, V. Jaqen: A high-performance switch-native approach for detecting and mitigating volumetric ddos attacks with programmable switches. In *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021* (2021), M. D. Bailey and R. Greenstadt, Eds., USENIX Association, pp. 3829–3846.

[22] METWALLY, A., AGRAWAL, D., AND ABBADI, A. E. Efficient computation of frequent and top-k elements in data streams. In *Database Theory - ICDT 2005, 10th International Conference, Edinburgh, UK, January 5-7, 2005, Proceedings* (2005), T. Eiter and L. Libkin, Eds., vol. 3363 of *Lecture Notes in Computer Science*, Springer, pp. 398–412.

[23] MISRA, J., AND GRIES, D. Finding repeated elements. *Sci. Comput. Program. 2*, 2 (1982), 143–152.

[24] SIVARAMAN, V., NARAYANA, S., ROTTENSTREICH, O., MUTHUKRISHNAN, S., AND REXFORD, J. Heavy-hitter detection entirely in the data plane. In *Proceedings of the Symposium on SDN Research, SOSR 2017, Santa Clara, CA, USA, April 3-4, 2017* (2017), ACM, pp. 164–176.

[25] ZHANG, M., LI, G., WANG, S., LIU, C., CHEN, A., HU, H., GU, G., LI, Q., XU, M., AND WU, J. Poseidon: Mitigating volumetric ddos attacks with programmable switches. In *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020* (2020), The Internet Society.

6.1.2 בחירת משתנה THRESHOLD 44
6.2 מציאת זרמים כבדים תוך התיחסות לזמן האחרון 45

7 סיכום 50

תוכן עניינים

1 מבוא 1

2 רקע 4

2.1 פתרונות קלאסיים 4

2.1.1 אלגוריתם של Misra Gries 5

2.1.2 האלגוריתם Space Saving 5

2.2 פתרונות במתגים ניתנים לתכנות 6

2.2.2 אלגוריתם HashPipe 8

2.2.3 אלגוריתם PRECISION 9

3 עיצוב RecenTo 10

3.1 סקירה של האלגוריתם 10

3.2 הצגת מבנה הנתונים של האלגוריתם 11

3.3 עדכון הרשומות 14

3.4 טיפול במופע כפול 19

3.5 הצעות נוספות לשיפור 21

4 מימוש 26

5 מימוש אמולציה 37

6 הערכה 39

6.1 הערכת דיוק האלגוריתם 40

6.1.1 בחירת משתנה Inc 40

תקציר

התקדמויות אחרונות ברשתות ניתנות לתכנות מאפשרות למפעילי רשת לבצע ניתוח דקדקני על כל התעבורה בעת חצייתה של הרשת. אחת המשימות הנפוצות ביותר, שנחקרה רבות, היא למצוא את הזרימות הכבדות ברשת (Top-K). מספר פתרונות הוצגו שמאתרים זרמים כאלה באמצעות כמות קבועה של זיכרון ופועלים במסגרת המשאבים והיכולות המוגבלות של מישור הנתונים. עם זאת, על מנת לזהות בעיות קריטיות ברשת, יש צורך לזהות את הזרמים שהיו כבדים בעבר הקרוב. ואולם, השיטות הקיימות מבצעות את משימת זיהוי הזרמים הכבדים באופן רציף, כך שהן יכולות למצוא את הזרמים הכבדים לאורך פרק זמן בלתי מוגבל. על מנת למצוא זרמים כבדים עבור פרקי זמן קצרים יותר, לעיתים קרובות משתמשים בחלונות הזזה. דבר זה דורש מופעים מרובים של המבנה כדי לנקות נתונים ישנים תוך כדי רישום נתונים חדשים. כך שזה דורש כמות גדולה של משאבים, וכן דורש קביעה מראש של אורך קבוע של החלונות. בעבודה זו אנו מספקים הגדרה פורמלית של זרמים כבדים אחרונים, ומציגים את RecenTo , אלגוריתם חדש דטרמיניסטי מבוסס מניה לאיתור זרמים כאלה במישור הנתונים. הטכניקה החדשנית שבה משתמש RecenTo מאפשרת לו 'לנקות את עצמו', וכך מקלה על זרמים חדשים להיכנס למבנה. RecenToגם משתמש במבנה חדשני לתחזוקת המפתח והמונה של הזרמים שמספק תמיכה לתלות ההדדית ביניהם)כלומר, לעיתים המונה משתנה בהתבסס על המפתח ולעיתים להפך(. אנו מעריכים את RecenTo על פרקי תעבורה רציפים ומראים שהוא משיג באופן עקבי שיעור זיהוי של כ-9.0 לאיתור הזרמים הכבדים האחרונים לכל פרקי התעבורה.

האוניברסיטה הפתוחה המחלקה למתמטיקה ולמדעי המחשב

זיהוי זרימות כבדות במישור הנתונים עם ובלי דגש על העבר הקרוב

פרויקט זה הוגש כחלק מהדרישות לקבלת תואר
"מוסמך למדעים" M.Sc. במדעי המחשב
באוניברסיטה הפתוחה
המחלקה למתמטיקה ומדעי המחשב

על־ידי
עוזרי אביה

בהנחיית שיר לנדאו פיביש

אוקטובר 2024